TP N°5 – SQL et triggers

Matisse Kra

BTS SIO 2e année

SLAM3 – Conception de BD

Sommaire

Introduction **2**
Exercice 1 : Parc immobilier **2**
Exercice 2 : Des produits au prix variant au fil du temps **17**
Exercice 3 : Location de véhicules **27**
Conclusion **34**

Introduction

Ce TP porte sur les triggers (ou déclencheurs) sous MySQL. Un trigger est un bloc de code SQL attaché à une table, que le serveur exécute automatiquement à chaque INSERT, UPDATE ou DELETE sur cette table. Il permet de faire respecter des règles de gestion que les clés primaires, les clés étrangères et les contraintes CHECK ne savent pas exprimer, par exemple une règle qui dépend d'une autre table ou d'une valeur calculée.

Un trigger se déclenche soit avant l'opération (BEFORE), soit après (AFTER). En BEFORE, on peut encore vérifier la ligne et la modifier, par exemple forcer une valeur, avant qu'elle soit écrite. En AFTER, la ligne a déjà passé tous les contrôles, ce qui en fait le bon moment pour répercuter le changement sur une autre table. Dans le corps du trigger, NEW désigne la ligne telle qu'elle va être insérée ou modifiée, et OLD la ligne avant la modification ou la suppression. Pour refuser une opération, on lève une erreur avec SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '...' : l'instruction est alors annulée et le client reçoit l'erreur 1644 avec le message choisi. Sous MySQL, un trigger ne porte que sur un seul événement : il faut un trigger pour l'insertion et un autre pour la mise à jour, même quand le contrôle est identique.

J'ai travaillé avec le client mysql de XAMPP. En affichant SELECT VERSION(), j'ai constaté que le serveur qui répondait était en fait un MySQL 9.7.4 installé par Homebrew : le socket de XAMPP est un lien vers celui de ce serveur. Cette version accepte plusieurs triggers par table, moment et événement, mais j'ai tout de même suivi la démarche du sujet : quand une question demande d'adapter un trigger, je le supprime puis je le recrée avec les nouveaux contrôles. J'ai créé une base par exercice (tp5_immobilier, tp5_commandes, tp5_location) et, pour chacun, quatre scripts rejouables depuis zéro : création des tables, triggers, jeu de données et tests. Chaque test exécute un SELECT avant et après l'opération, pour montrer son effet ou l'absence d'effet quand le trigger la refuse. Les trois exercices sont présentés ci-dessous avec le code des triggers et les captures des tests.

Exercice 1 : Parc immobilier

Création des tables et jeu de données

J'ai recopié tel quel le script de création fourni par le sujet, en gardant la clé primaire de Photo sur (immeuble, appartement, reference). Le script commence par DROP DATABASE IF EXISTS pour pouvoir être rejoué de zéro.

ex1/01_creation_tables.sql — MySQL 9.7.4 – base tp5_immobilier

Ex. 1 – Script de création des tables (script du sujet)

```sql
DROP DATABASE IF EXISTS tp5_immobilier;
CREATE DATABASE tp5_immobilier CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE tp5_immobilier;

CREATE TABLE Immeuble(
    Id INT(11) NOT NULL AUTO_INCREMENT PRIMARY KEY,
    adrNum VARCHAR(7) NOT NULL, adrVoie VARCHAR(100) NOT NULL,
    adrCodePostal VARCHAR(5) NOT NULL, adrVille VARCHAR(30) NOT NULL,
    fibreOptique TINYINT NOT NULL, parkingPrivatif TINYINT NOT NULL
);

CREATE TABLE Appartement(
    immeuble INT(11), num INT(3) NOT NULL, description LONGTEXT,
    loyer DOUBLE NOT NULL, superficie DOUBLE NOT NULL,
    terrasse TINYINT(1) NOT NULL, classeConso CHAR(1) NOT NULL, chauffage CHAR(1) NOT NULL,
    placeParking TINYINT(1) NOT NULL, prixParking DOUBLE,
    CONSTRAINT pk_appartement PRIMARY KEY (immeuble, num),
    CONSTRAINT fk_immeuble FOREIGN KEY (immeuble) REFERENCES Immeuble(id)
);

CREATE TABLE Photo(
    immeuble INT(11), appartement INT(3), reference INT(11) NOT NULL,
    titre VARCHAR(75), description VARCHAR(255), uri VARCHAR(120) NOT NULL,
    CONSTRAINT pk_photo PRIMARY KEY (immeuble, appartement, reference),
    CONSTRAINT fk_appartement_photo
        FOREIGN KEY (immeuble, appartement) REFERENCES Appartement(immeuble, num)
);

CREATE TABLE Piece(
    immeuble INT(11), appartement INT(3), num INT(2) NOT NULL,
    superficie DOUBLE ,
    fonction VARCHAR(30),
    CONSTRAINT pk_piece PRIMARY KEY (immeuble, appartement, num),
    CONSTRAINT fk_appartement_piece
        FOREIGN KEY (immeuble, appartement) REFERENCES Appartement(immeuble, num)
);
```

Script de création des tables de l'exercice 1

Le jeu de données comporte trois immeubles : ceux de Nantes et de Rennes ont un parking privatif, celui d'Angers n'en a pas. Il contient aussi cinq appartements et leurs pièces. Je l'insère après avoir créé les triggers, ce qui fait que les appartements sont créés avec une superficie de 0 et que ce sont les pièces qui la calculent au fur et à mesure (questions 2 et 3).

ex1/03_jeu_de_donnees.sql — MySQL 9.7.4 – base tp5_immobilier

Ex. 1 – Jeu de données

```sql
-- Immeubles : 1 et 3 avec parking privatif, 2 sans parking
INSERT INTO Immeuble (Id, adrNum, adrVoie, adrCodePostal, adrVille, fibreOptique, parkingPrivatif) VALUES
    (1, '12',    'rue des Lilas',             '44000', 'Nantes', 1, 1),
    (2, '3 bis', 'avenue Jean Jaurès',        '49000', 'Angers', 0, 0),
    (3, '27',    'boulevard de la Liberté',   '35000', 'Rennes', 1, 1);

-- Appartements (superficie à 0 : elle est calculée par les triggers)
INSERT INTO Appartement (immeuble, num, description, loyer, superficie, terrasse,
                         classeConso, chauffage, placeParking, prixParking) VALUES
    (1, 1, 'T2 lumineux, proche tram',     620, 0, 0, 'C', 'E', 1, 45),
    (1, 2, 'T3 avec terrasse plein sud',   850, 0, 1, 'B', 'G', 0, NULL),
    (2, 1, 'Studio centre-ville',          430, 0, 0, 'D', 'E', 0, NULL),
    (2, 2, 'T2 rénové',                    560, 0, 0, 'C', 'G', 0, NULL),
    (3, 1, 'T4 familial',                  980, 0, 1, 'B', 'G', 1, 60);

-- Pièces
INSERT INTO Piece (immeuble, appartement, num, superficie, fonction) VALUES
    (1, 1, 1, 22.5, 'Séjour'),
    (1, 1, 2, 11,   'Chambre'),
    (1, 1, 3, 6.5,  'Cuisine'),
    (1, 1, 4, 4,    'Salle de bain'),
    (1, 2, 1, 28,   'Séjour'),
    (1, 2, 2, 12,   'Chambre'),
    (1, 2, 3, 10.5, 'Chambre'),
    (1, 2, 4, 8,    'Cuisine'),
    (2, 1, 1, 18,   'Pièce principale'),
    (2, 1, 2, 3.5,  'Salle d''eau'),
    (2, 2, 1, 20,   'Séjour'),
    (2, 2, 2, 10,   'Chambre'),
    (3, 1, 1, 32,   'Séjour'),
    (3, 1, 2, 13,   'Chambre'),
    (3, 1, 3, 11,   'Chambre'),
    (3, 1, 4, 10,   'Chambre'),
    (3, 1, 5, 9,    'Cuisine');

-- Photos
INSERT INTO Photo (immeuble, appartement, reference, titre, description, uri) VALUES
    (1, 1, 1, 'Séjour',   'Vue du séjour',          'photos/1-1-sejour.jpg'),
    (1, 2, 1, 'Terrasse', 'Terrasse plein sud',     'photos/1-2-terrasse.jpg'),
    (3, 1, 1, 'Façade',   'Façade de la résidence', 'photos/3-1-facade.jpg');
```

Jeu de données de l'exercice 1

Question 1 : prix de la place de parking

La règle est la suivante : prixParking doit être NULL quand placeParking vaut 0, et renseigné quand placeParking vaut 1. J'ai écrit deux triggers BEFORE, l'un sur INSERT et l'autre sur UPDATE, avec les deux mêmes tests. J'ai choisi BEFORE parce que la ligne doit être refusée avant d'être écrite. Chaque cas d'erreur a son propre message, pour que l'utilisateur sache tout de suite ce qu'il doit corriger.

ex1/02_triggers.sql — MySQL 9.7.4 – base tp5_immobilier

Q1 – Prix de la place de parking (BEFORE INSERT et BEFORE UPDATE)

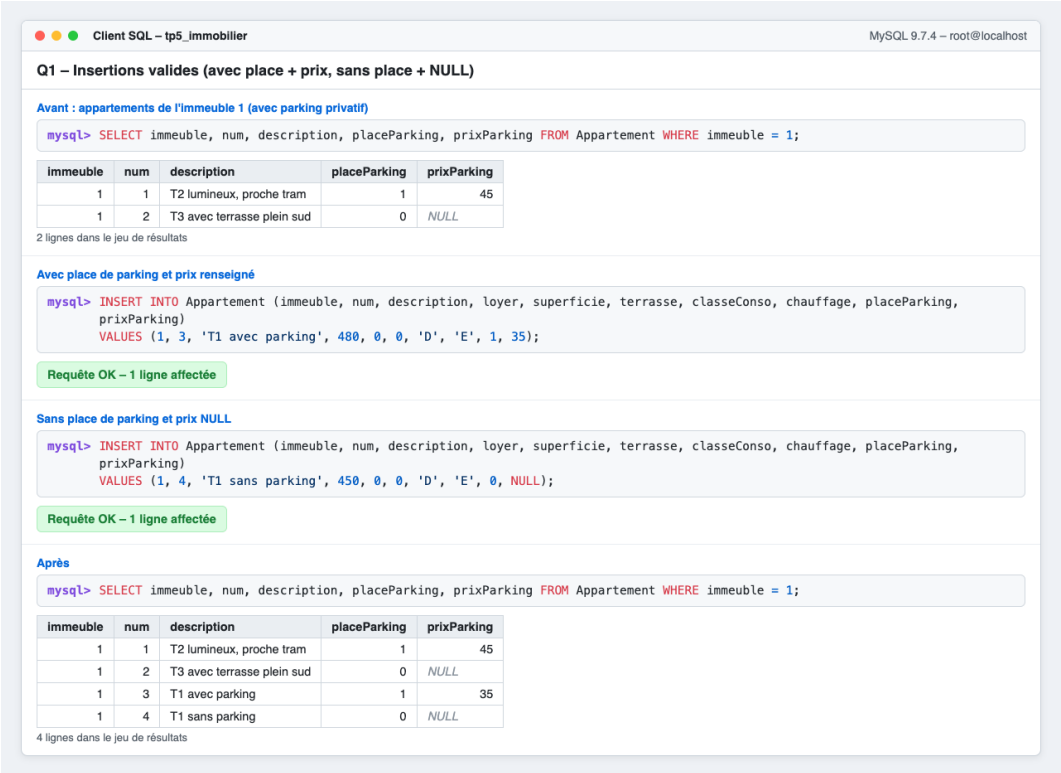
```sql
-- ------------------------------------------------------------
-- Question 1 : prixParking doit être NULL si placeParking = 0,
-- et renseigné si placeParking = 1. Un trigger MySQL ne porte que
-- sur un seul événement : un trigger pour INSERT, un pour UPDATE.
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_appartement_bi;
DROP TRIGGER IF EXISTS trg_appartement_bu;

DELIMITER //
CREATE TRIGGER trg_appartement_bi
BEFORE INSERT ON Appartement
FOR EACH ROW
BEGIN
    IF NEW.placeParking = 0 AND NEW.prixParking IS NOT NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Pas de place de parking : le prix du parking doit être NULL.';
    END IF;
    IF NEW.placeParking = 1 AND NEW.prixParking IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Place de parking : le prix du parking doit être renseigné.';
    END IF;
END //

CREATE TRIGGER trg_appartement_bu
BEFORE UPDATE ON Appartement
FOR EACH ROW
BEGIN
    IF NEW.placeParking = 0 AND NEW.prixParking IS NOT NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Pas de place de parking : le prix du parking doit être NULL.';
    END IF;
    IF NEW.placeParking = 1 AND NEW.prixParking IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Place de parking : le prix du parking doit être renseigné.';
    END IF;
END //
DELIMITER ;
```

Triggers trg_appartement_bi et trg_appartement_bu (question 1)

Pour tester, j'ai d'abord inséré deux appartements valides dans l'immeuble 1 : le 1-3 avec une place à 35 € et le 1-4 sans place, avec un prix NULL. Les deux insertions sont acceptées et apparaissent dans le SELECT après.



Insertions valides acceptées

J'ai ensuite tenté d'insérer l'appartement 1-5 de deux façons incorrectes : sans place mais avec un prix de 50 €, puis avec une place mais sans prix. Les deux fois, le trigger lève l'erreur 1644 avec le message correspondant, et le SELECT qui suit montre que l'appartement n'existe pas.

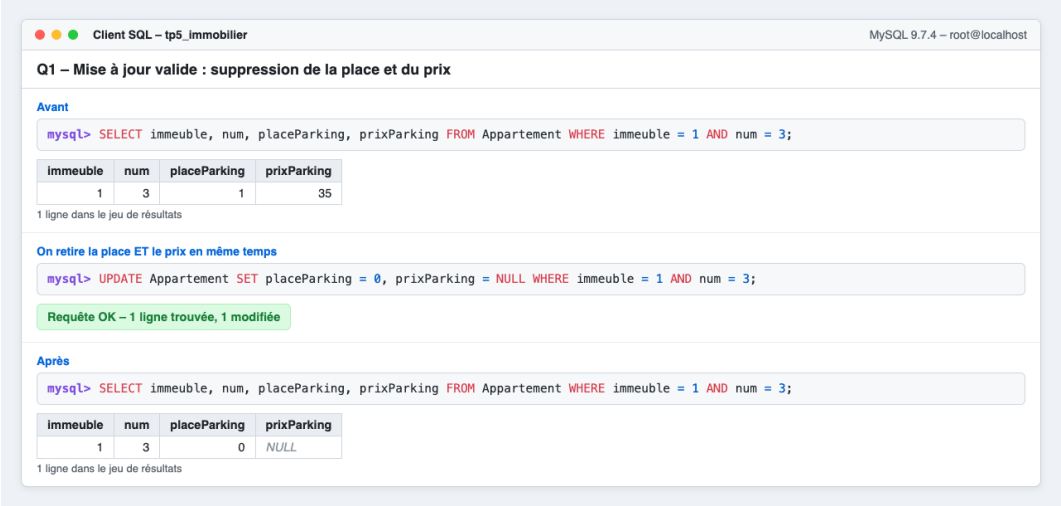


Insertion refusée : prix renseigné sans place de parking

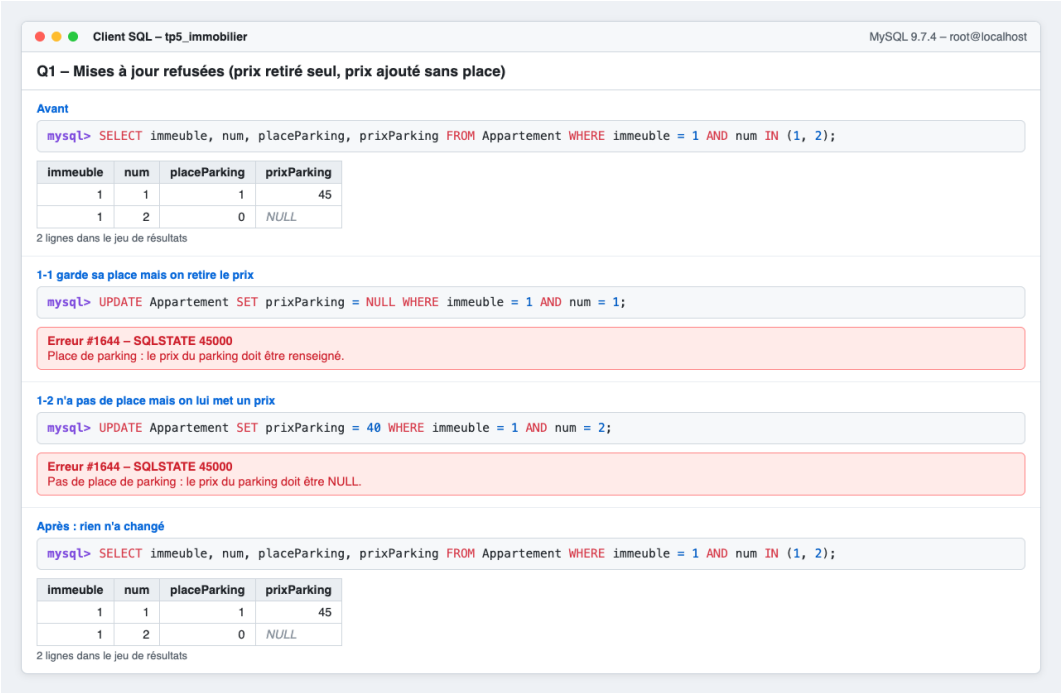


Insertion refusée : place de parking sans prix

J'ai fait les mêmes vérifications en mise à jour. Retirer la place et le prix dans le même UPDATE est accepté. Retirer seulement le prix d'un appartement qui garde sa place, ou mettre un prix à un appartement qui n'en a pas, est refusé, et les valeurs restent celles d'avant.



Mise à jour valide : place et prix retirés ensemble



Mises à jour refusées par trg_appartement_bu

Question 2 : superficie mise à jour à l'insertion d'une pièce

La superficie d'un appartement doit être égale à la somme des superficies de ses pièces. J'ai écrit un trigger AFTER INSERT sur Piece qui ajoute la superficie de la nouvelle pièce à celle de son appartement. J'ai pris AFTER plutôt que BEFORE pour que la pièce ait déjà passé la clé primaire et la clé étrangère avant qu'on touche à l'appartement. J'ai encadré NEW.superficie par COALESCE car la colonne superficie de Piece peut être NULL, et superficie + NULL donnerait NULL, ce qui effacerait la superficie de l'appartement.

ex1/02_triggers.sql — MySQL 9.7.4 – base tp5_immobilier

Q2 – Ajout de la superficie d'une pièce à l'appartement (AFTER INSERT)

```sql
-- ------------------------------------------------------------
-- Question 2 : à l'insertion d'une pièce, sa superficie est
-- ajoutée à celle de l'appartement. AFTER : la pièce a passé
-- tous les contrôles (clé primaire, clé étrangère) avant qu'on
-- touche à l'appartement. COALESCE car Piece.superficie peut
-- être NULL, et superficie + NULL donnerait NULL.
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_piece_ai;

DELIMITER //
CREATE TRIGGER trg_piece_ai
AFTER INSERT ON Piece
FOR EACH ROW
BEGIN
    UPDATE Appartement
       SET superficie = superficie + COALESCE(NEW.superficie, 0)
     WHERE immeuble = NEW.immeuble
       AND num = NEW.appartement;
END //
DELIMITER ;
```

Trigger trg_piece_ai (question 2)

Dans le test, l'appartement 1-3, qui venait d'être créé, avait une superficie de 0. Après l'ajout d'une pièce de 25 m² et d'une salle d'eau de 4,5 m², il mesure 29,5 m². J'ai aussi vérifié le cas d'une insertion qui échoue : la pièce 1-3-1 existe déjà, la clé primaire refuse le doublon (erreur 1062) et le trigger AFTER ne s'exécute pas, donc la superficie reste à 29,5.

Client SQL – tp5_immobilier — MySQL 9.7.4 – root@localhost

Q2 – Insertion de pièces : la superficie de l'appartement augmente

Avant : l'appartement 1-3 vient d'être créé, superficie 0

```
mysql> SELECT immeuble, num, superficie FROM Appartement WHERE immeuble = 1 AND num = 3;
```

immeuble	num	superficie
1	3	0

1 ligne dans le jeu de résultats

```
mysql> INSERT INTO Piece (immeuble, appartement, num, superficie, fonction) VALUES (1, 3, 1, 25, 'Pièce principale');
```

Requête OK – 1 ligne affectée

```
mysql> INSERT INTO Piece (immeuble, appartement, num, superficie, fonction) VALUES (1, 3, 2, 4.5, 'Salle d''eau');
```

Requête OK – 1 ligne affectée

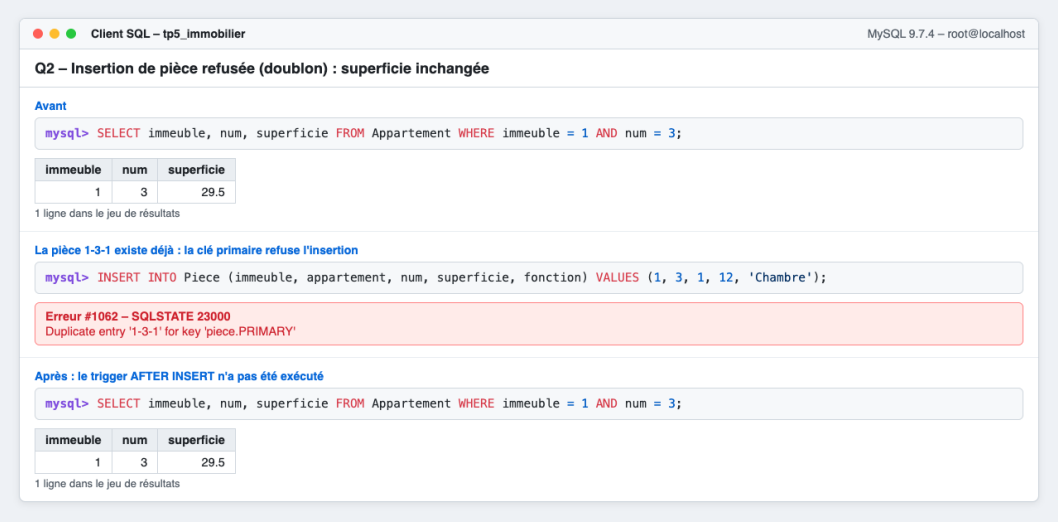
Après : 0 + 25 + 4,5 = 29,5

```
mysql> SELECT immeuble, num, superficie FROM Appartement WHERE immeuble = 1 AND num = 3;
```

immeuble	num	superficie
1	3	29.5

1 ligne dans le jeu de résultats

La superficie de l'appartement augmente à chaque pièce insérée



Insertion d'une pièce en doublon : superficie inchangée

Question 3 : trigger de la question 1 adapté

J'ai repris les deux triggers de la question 1 pour leur ajouter trois règles. D'abord, un appartement ne peut avoir de place de parking que si son immeuble a un parking privatif : quand placeParking vaut 1, je lis parkingPrivatif dans Immeuble avec SELECT INTO et je refuse la ligne si la valeur est 0. Ensuite, à l'insertion, je force NEW.superficie à 0, puisque ce sont les pièces qui vont la construire.

ex1/02_triggers.sql — MySQL 9.7.4 – base tp5_immobilier

Q3 – Trigger BEFORE INSERT sur Appartement adapté

```sql
-- ------------------------------------------------------------
-- Question 3 (insertion) : on reprend le trigger de la Q1 et on
-- ajoute :
--  - refus d'une place de parking si l'immeuble n'a pas de
--    parking privatif ;
--  - superficie forcée à 0 : elle sera calculée par les
--    triggers de Piece au fur et à mesure des pièces ajoutées.
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_appartement_bi;

DELIMITER //
CREATE TRIGGER trg_appartement_bi
BEFORE INSERT ON Appartement
FOR EACH ROW
BEGIN
    DECLARE v_parking_privatif TINYINT;

    -- (Q1) cohérence placeParking / prixParking
    IF NEW.placeParking = 0 AND NEW.prixParking IS NOT NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Pas de place de parking : le prix du parking doit être NULL.';
    END IF;
    IF NEW.placeParking = 1 AND NEW.prixParking IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Place de parking : le prix du parking doit être renseigné.';
    END IF;

    -- (Q3) pas de place de parking sans parking privatif
    IF NEW.placeParking = 1 THEN
        SELECT parkingPrivatif INTO v_parking_privatif
          FROM Immeuble
         WHERE id = NEW.immeuble;
        IF v_parking_privatif = 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Place de parking refusée : l''immeuble n''a pas de parking privatif.';
        END IF;
    END IF;

    -- (Q3) superficie initialisée à 0
    SET NEW.superficie = 0;
END //
DELIMITER ;
```

Trigger trg_appartement_bi adapté (question 3)

Enfin, la superficie ne doit plus pouvoir être modifiée par un UPDATE sur Appartement. Le sujet laisse deux possibilités : remettre la valeur à OLD.superficie sans rien dire, ou lever une erreur. J'ai choisi l'erreur. Avec une remise silencieuse, l'utilisateur recevrait « Requête OK » et croirait que sa modification a été enregistrée. La contrepartie est qu'un UPDATE qui modifie le loyer et la superficie en même temps est refusé en entier. Je compare les deux valeurs avec <=>, l'égalité qui tient compte de NULL, pour ne bloquer que si la superficie change vraiment :

```
IF NOT (NEW.superficie <=> OLD.superficie)
  AND COALESCE(@maj_superficie_piece, 0) <> 1 THEN
```

ex1/02_triggers.sql — MySQL 9.7.4 – base tp5_immobilier

Q3 – Trigger BEFORE UPDATE sur Appartement adapté (superficie bloquée)

```sql
-- ------------------------------------------------------------
-- Question 3 (mise à jour) : mêmes contrôles de parking, et la
-- superficie ne peut plus être modifiée directement (erreur).
--
-- CONFLIT avec les triggers de Piece (Q2 et Q4) : eux aussi
-- modifient la superficie par un UPDATE Appartement, qui
-- déclencherait ce blocage. Solution : les triggers de Piece
-- positionnent la variable de session @maj_superficie_piece à 1
-- juste avant leur UPDATE et la remettent à NULL juste après.
-- Ce trigger laisse passer la modification seulement dans ce cas.
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_appartement_bu;

DELIMITER //
CREATE TRIGGER trg_appartement_bu
BEFORE UPDATE ON Appartement
FOR EACH ROW
BEGIN
    DECLARE v_parking_privatif TINYINT;

    -- (Q1) cohérence placeParking / prixParking
    IF NEW.placeParking = 0 AND NEW.prixParking IS NOT NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Pas de place de parking : le prix du parking doit être NULL.';
    END IF;
    IF NEW.placeParking = 1 AND NEW.prixParking IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Place de parking : le prix du parking doit être renseigné.';
    END IF;

    -- (Q3) pas de place de parking sans parking privatif
    IF NEW.placeParking = 1 THEN
        SELECT parkingPrivatif INTO v_parking_privatif
          FROM Immeuble
         WHERE id = NEW.immeuble;
        IF v_parking_privatif = 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Place de parking refusée : l''immeuble n''a pas de parking privatif.';
        END IF;
    END IF;

    -- (Q3) superficie non modifiable, sauf par les triggers de Piece
    IF NOT (NEW.superficie <=> OLD.superficie)
       AND COALESCE(@maj_superficie_piece, 0) <> 1 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Modification interdite : la superficie est calculée à partir des pièces.';
    END IF;
END //
DELIMITER ;
```

Trigger trg_appartement_bu adapté (question 3)

Les tests montrent les trois règles. Dans l'immeuble 2, qui n'a pas de parking privatif, l'insertion d'un appartement avec une place et la mise à jour d'un appartement existant pour lui en donner une sont refusées. Un appartement inséré avec une superficie de 120 m² est enregistré avec 0. Enfin, un UPDATE de la superficie de l'appartement 1-1 est refusé, alors que le passage du loyer de 620 à 640 € est accepté.

Client SQL – tp5_immobilier — MySQL 9.7.4 – root@localhost

Q3 – Place de parking refusée dans un immeuble sans parking privatif

L'immeuble 2 n'a pas de parking privatif

```
mysql> SELECT Id, adrNum, adrVoie, adrVille, parkingPrivatif FROM Immeuble WHERE Id = 2;
```

Id	adrNum	adrVoie	adrVille	parkingPrivatif
2	3 bis	avenue Jean Jaurès	Angers	0

1 ligne dans le jeu de résultats

Insertion d'un appartement avec place de parking

```
mysql> INSERT INTO Appartement (immeuble, num, description, loyer, superficie, terrasse, classeConso, chauffage, placeParking,
       prixParking)
       VALUES (2, 3, 'T2 avec parking', 590, 0, 0, 'C', 'G', 1, 40);
```

Erreur #1644 – SQLSTATE 45000
Place de parking refusée : l'immeuble n'a pas de parking privatif.

Mise à jour d'un appartement existant pour lui donner une place

```
mysql> UPDATE Appartement SET placeParking = 1, prixParking = 40 WHERE immeuble = 2 AND num = 1;
```

Erreur #1644 – SQLSTATE 45000
Place de parking refusée : l'immeuble n'a pas de parking privatif.

Après : rien n'a été inséré ni modifié

```
mysql> SELECT immeuble, num, placeParking, prixParking FROM Appartement WHERE immeuble = 2;
```

immeuble	num	placeParking	prixParking
2	1	0	*NULL*
2	2	0	*NULL*

2 lignes dans le jeu de résultats

Place de parking refusée dans un immeuble sans parking privatif

Client SQL – tp5_immobilier — MySQL 9.7.4 – root@localhost

Q3 – Superficie initialisée à 0 à l'insertion d'un appartement

On insère volontairement une superficie de 120 m²

```
mysql> INSERT INTO Appartement (immeuble, num, description, loyer, superficie, terrasse, classeConso, chauffage, placeParking,
       prixParking)
       VALUES (2, 3, 'T3 sans parking', 690, 120, 0, 'C', 'G', 0, NULL);
```

Requête OK – 1 ligne affectée

Après : le trigger a remis la superficie à 0

```
mysql> SELECT immeuble, num, description, superficie FROM Appartement WHERE immeuble = 2 AND num = 3;
```

immeuble	num	description	superficie
2	3	T3 sans parking	0

1 ligne dans le jeu de résultats

Superficie forcée à 0 à l'insertion

Client SQL – tp5_immobilier — MySQL 9.7.4 – root@localhost

Q3 – Modification directe de la superficie refusée

Avant

```
mysql> SELECT immeuble, num, loyer, superficie FROM Appartement WHERE immeuble = 1 AND num = 1;
```

immeuble	num	loyer	superficie
1	1	620	44

1 ligne dans le jeu de résultats

Modification directe de la superficie

```
mysql> UPDATE Appartement SET superficie = 200 WHERE immeuble = 1 AND num = 1;
```

Erreur #1644 – SQLSTATE 45000
Modification interdite : la superficie est calculée à partir des pièces.

Après : superficie inchangée

```
mysql> SELECT immeuble, num, loyer, superficie FROM Appartement WHERE immeuble = 1 AND num = 1;
```

immeuble	num	loyer	superficie
1	1	620	44

1 ligne dans le jeu de résultats

Modification directe de la superficie refusée

Client SQL – tp5_immobilier — MySQL 9.7.4 – root@localhost

Q3 – Mise à jour d'une autre colonne autorisée

Avant

```
mysql> SELECT immeuble, num, loyer, superficie FROM Appartement WHERE immeuble = 1 AND num = 1;
```

immeuble	num	loyer	superficie
1	1	620	44

1 ligne dans le jeu de résultats

Le loyer peut toujours être modifié (superficie non touchée)

```
mysql> UPDATE Appartement SET loyer = 640 WHERE immeuble = 1 AND num = 1;
```

Requête OK – 1 ligne trouvée, 1 modifiée

Après

```
mysql> SELECT immeuble, num, loyer, superficie FROM Appartement WHERE immeuble = 1 AND num = 1;
```

immeuble	num	loyer	superficie
1	1	640	44

1 ligne dans le jeu de résultats

Mise à jour du loyer acceptée

Question 4 : mise à jour et suppression d'une pièce

Avant d'écrire ces triggers, il fallait régler un conflit. Le trigger de la question 2, comme ceux de la question 4, modifie la superficie par un UPDATE Appartement. Cet UPDATE déclenche trg_appartement_bu, qui le refuserait depuis la question 3. J'ai résolu ce conflit avec une variable de session : chaque trigger de Piece met @maj_superficie_piece à 1 juste avant son UPDATE et la remet à NULL juste après. Le trigger d'Appartement ne laisse passer un changement de superficie que si cette variable vaut 1. J'ai donc recréé le trigger de la question 2 sur ce modèle. Cette protection empêche les erreurs de manipulation mais ne protège pas d'un utilisateur malveillant : quelqu'un qui tape lui-même SET @maj_superficie_piece = 1 contourne le blocage.

ex1/02_triggers.sql — MySQL 9.7.4 – base tp5_immobilier

Q3/Q4 – Résolution du conflit : trigger de la Q2 adapté

```
1  -- ------------------------------------------------------------
2  -- Résolution du conflit Q3 / Q2 : le trigger d'insertion de
3  -- pièce est recréé pour « s'annoncer » avant de modifier la
4  -- superficie de l'appartement.
5  -- ------------------------------------------------------------
6  DROP TRIGGER IF EXISTS trg_piece_ai;
7
8  DELIMITER //
9  CREATE TRIGGER trg_piece_ai
10 AFTER INSERT ON Piece
11 FOR EACH ROW
12 BEGIN
13     SET @maj_superficie_piece = 1;     -- autorise la mise à jour
14     UPDATE Appartement
15        SET superficie = superficie + COALESCE(NEW.superficie, 0)
16      WHERE immeuble = NEW.immeuble
17        AND num = NEW.appartement;
18     SET @maj_superficie_piece = NULL;  -- referme l'autorisation
19 END //
20 DELIMITER ;
```

Trigger trg_piece_ai recréé avec la variable de session

Le trigger AFTER UPDATE sur Piece distingue deux cas. Si la pièce reste dans le même appartement, la superficie devient superficie + NEW.superficie - OLD.superficie. Si la pièce change d'appartement, je retire OLD.superficie à l'ancien appartement, identifié par OLD.immeuble et OLD.appartement, et j'ajoute NEW.superficie au nouveau. Sans ce second cas, l'ancien appartement garderait la surface d'une pièce qu'il n'a plus. Le trigger AFTER DELETE retire simplement OLD.superficie.

ex1/02_triggers.sql — MySQL 9.7.4 – base tp5_immobilier

Q4 – Mise à jour d'une pièce (AFTER UPDATE)

```sql
-- ------------------------------------------------------------
-- Question 4 (mise à jour d'une pièce) :
--  - même appartement : superficie + NEW - OLD ;
--  - la pièce change d'appartement : on retire OLD.superficie à
--    l'ancien appartement et on ajoute NEW.superficie au nouveau.
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_piece_au;

DELIMITER //
CREATE TRIGGER trg_piece_au
AFTER UPDATE ON Piece
FOR EACH ROW
BEGIN
    SET @maj_superficie_piece = 1;
    IF NEW.immeuble = OLD.immeuble AND NEW.appartement = OLD.appartement THEN
        UPDATE Appartement
           SET superficie = superficie + COALESCE(NEW.superficie, 0)
                                       - COALESCE(OLD.superficie, 0)
         WHERE immeuble = NEW.immeuble
           AND num = NEW.appartement;
    ELSE
        UPDATE Appartement
           SET superficie = superficie - COALESCE(OLD.superficie, 0)
         WHERE immeuble = OLD.immeuble
           AND num = OLD.appartement;
        UPDATE Appartement
           SET superficie = superficie + COALESCE(NEW.superficie, 0)
         WHERE immeuble = NEW.immeuble
           AND num = NEW.appartement;
    END IF;
    SET @maj_superficie_piece = NULL;
END //
DELIMITER ;
```

Trigger trg_piece_au (question 4)

ex1/02_triggers.sql — MySQL 9.7.4 – base tp5_immobilier

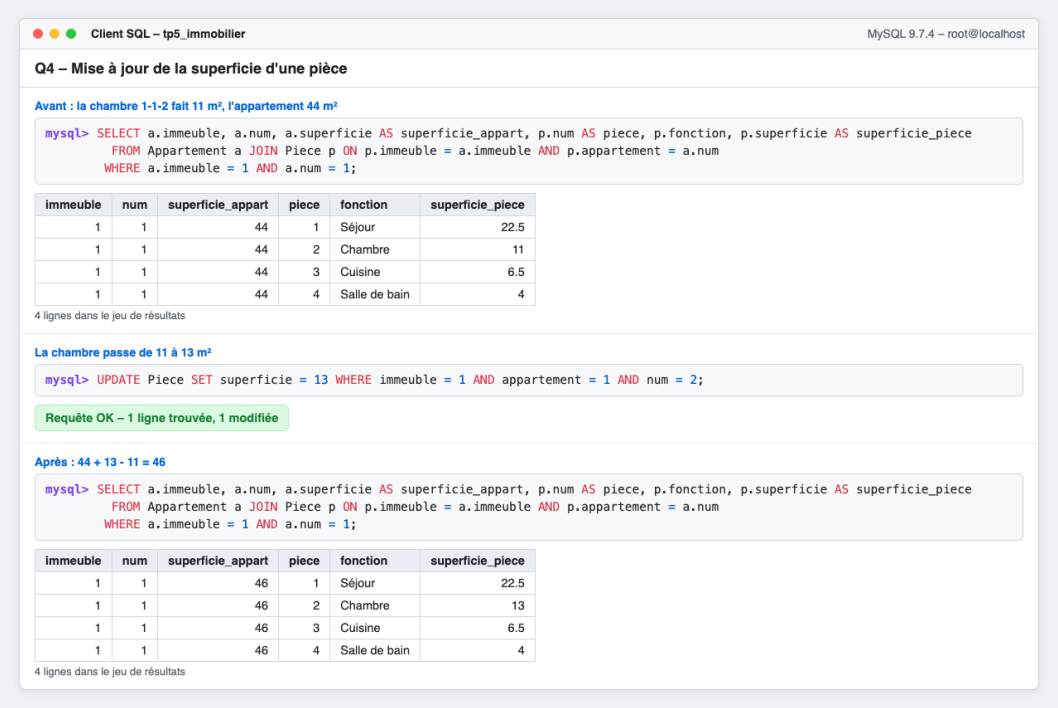
Q4 – Suppression d'une pièce (AFTER DELETE)

```sql
-- ------------------------------------------------------------
-- Question 4 (suppression d'une pièce) : superficie - OLD.
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_piece_ad;

DELIMITER //
CREATE TRIGGER trg_piece_ad
AFTER DELETE ON Piece
FOR EACH ROW
BEGIN
    SET @maj_superficie_piece = 1;
    UPDATE Appartement
       SET superficie = superficie - COALESCE(OLD.superficie, 0)
     WHERE immeuble = OLD.immeuble
       AND num = OLD.appartement;
    SET @maj_superficie_piece = NULL;
END //
DELIMITER ;
```

Trigger trg_piece_ad (question 4)

Quand la chambre 1-1-2 passe de 11 à 13 m², l'appartement passe de 44 à 46 m². Quand la salle d'eau de 4,5 m² est rattachée à l'appartement 2-3, l'appartement 1-3 passe de 29,5 à 25 m² et le 2-3 de 0 à 4,5 m². La suppression de la salle de bain de 4 m² ramène l'appartement 1-1 à 42 m².



Client SQL – tp5_immobilier — MySQL 9.7.4 – root@localhost

Q4 – Mise à jour de la superficie d'une pièce

Avant : la chambre 1-1-2 fait 11 m², l'appartement 44 m²

```
mysql> SELECT a.immeuble, a.num, a.superficie AS superficie_appart, p.num AS piece, p.fonction, p.superficie AS superficie_piece
         FROM Appartement a JOIN Piece p ON p.immeuble = a.immeuble AND p.appartement = a.num
        WHERE a.immeuble = 1 AND a.num = 1;
```

immeuble	num	superficie_appart	piece	fonction	superficie_piece
1	1	44	1	Séjour	22.5
1	1	44	2	Chambre	11
1	1	44	3	Cuisine	6.5
1	1	44	4	Salle de bain	4

4 lignes dans le jeu de résultats

La chambre passe de 11 à 13 m²

```
mysql> UPDATE Piece SET superficie = 13 WHERE immeuble = 1 AND appartement = 1 AND num = 2;
```

Requête OK – 1 ligne trouvée, 1 modifiée

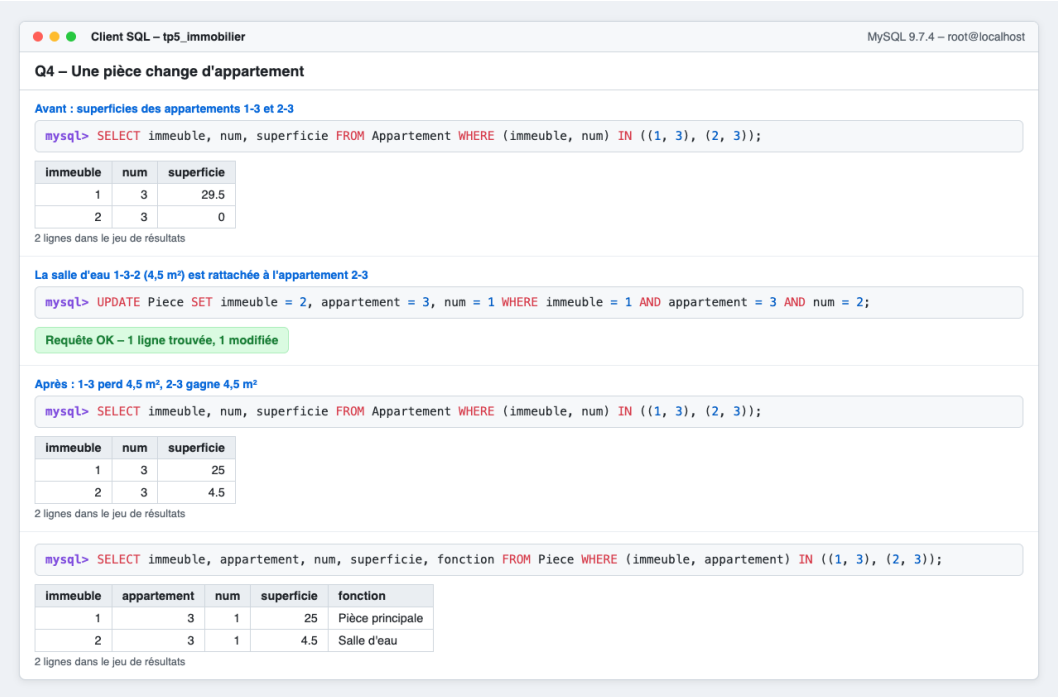
Après : 44 + 13 - 11 = 46

```
mysql> SELECT a.immeuble, a.num, a.superficie AS superficie_appart, p.num AS piece, p.fonction, p.superficie AS superficie_piece
         FROM Appartement a JOIN Piece p ON p.immeuble = a.immeuble AND p.appartement = a.num
        WHERE a.immeuble = 1 AND a.num = 1;
```

immeuble	num	superficie_appart	piece	fonction	superficie_piece
1	1	46	1	Séjour	22.5
1	1	46	2	Chambre	13
1	1	46	3	Cuisine	6.5
1	1	46	4	Salle de bain	4

4 lignes dans le jeu de résultats

Mise à jour de la superficie d'une pièce



Client SQL – tp5_immobilier — MySQL 9.7.4 – root@localhost

Q4 – Une pièce change d'appartement

Avant : superficies des appartements 1-3 et 2-3

```
mysql> SELECT immeuble, num, superficie FROM Appartement WHERE (immeuble, num) IN ((1, 3), (2, 3));
```

immeuble	num	superficie
1	3	29.5
2	3	0

2 lignes dans le jeu de résultats

La salle d'eau 1-3-2 (4,5 m²) est rattachée à l'appartement 2-3

```
mysql> UPDATE Piece SET immeuble = 2, appartement = 3, num = 1 WHERE immeuble = 1 AND appartement = 3 AND num = 2;
```

Requête OK – 1 ligne trouvée, 1 modifiée

Après : 1-3 perd 4,5 m², 2-3 gagne 4,5 m²

```
mysql> SELECT immeuble, num, superficie FROM Appartement WHERE (immeuble, num) IN ((1, 3), (2, 3));
```

immeuble	num	superficie
1	3	25
2	3	4.5

2 lignes dans le jeu de résultats

```
mysql> SELECT immeuble, appartement, num, superficie, fonction FROM Piece WHERE (immeuble, appartement) IN ((1, 3), (2, 3));
```

immeuble	appartement	num	superficie	fonction
1	3	1	25	Pièce principale
2	3	1	4.5	Salle d'eau

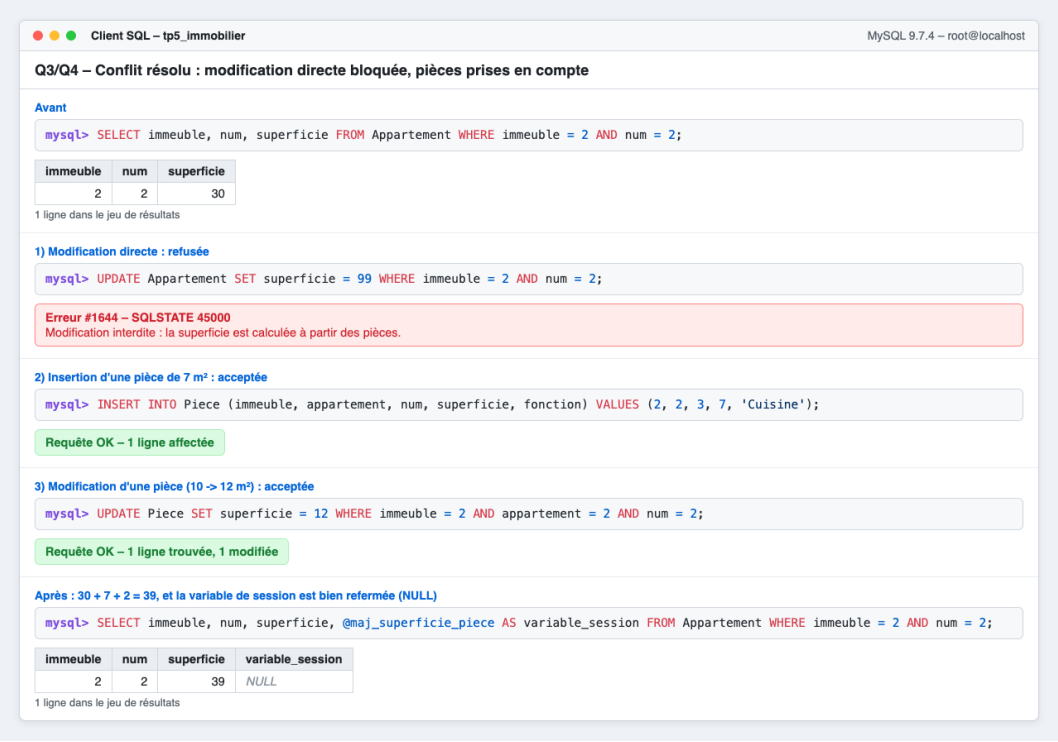
2 lignes dans le jeu de résultats

Une pièce change d'appartement



Suppression d'une pièce

Le dernier test vérifie la résolution du conflit sur l'appartement 2-2. La modification directe de sa superficie à 99 m² est refusée, mais l'ajout d'une cuisine de 7 m² et le passage d'une chambre de 10 à 12 m² sont acceptés : la superficie passe de 30 à 39 m² et la variable de session est bien revenue à NULL. Une requête de contrôle compare enfin, pour chaque appartement, la superficie avec la somme de ses pièces : les huit appartements sont tous à OK.



Conflit résolu : modification directe bloquée, pièces prises en compte



Vérification : superficie égale à la somme des pièces

Exercice 2 : Des produits au prix variant au fil du temps

Question 1 : création des tables

J'ai écrit le script de création à partir du schéma relationnel. Les prix sont en DECIMAL plutôt qu'en DOUBLE, pour éviter les arrondis sur des montants. Le champ formation vaut 1 pour une formation et 0 pour un livre, et le poids, en grammes, est NULL pour une formation. Le sujet écrit « livraion » dans Commande : c'est une coquille, j'ai nommé la colonne livraison. Les colonnes livraison et transport de Commande acceptent NULL, ce qui correspond à une commande sans frais de transport. La clé étrangère (livraison, transport) pointe vers Frais_Transport. Dans Ligne_Commande, prixu et prixt acceptent NULL parce qu'ils sont calculés par le trigger de la question 5. Une contrainte CHECK impose une quantité positive.

ex2/01_creation_tables.sql — MySQL 9.7.4 – base tp5_commandes

Q1 – Script de création des tables

```sql
DROP DATABASE IF EXISTS tp5_commandes;
CREATE DATABASE tp5_commandes CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE tp5_commandes;

CREATE TABLE Produit(
    code      INT NOT NULL AUTO_INCREMENT,
    libelle   VARCHAR(100) NOT NULL,
    prix      DECIMAL(8,2) NOT NULL,
    formation TINYINT(1) NOT NULL,   -- 1 = formation, 0 = livre
    poids     INT NULL,              -- en grammes (NULL pour une formation)
    CONSTRAINT pk_produit PRIMARY KEY (code)
);

CREATE TABLE Mode_Livraison(
    num     INT NOT NULL AUTO_INCREMENT,
    libelle VARCHAR(50) NOT NULL,
    CONSTRAINT pk_mode_livraison PRIMARY KEY (num)
);

CREATE TABLE Frais_Transport(
    livraison INT NOT NULL,
    num       INT NOT NULL,
    prix      DECIMAL(6,2) NOT NULL,
    poidsMin  INT NOT NULL,
    poidsMax  INT NOT NULL,
    CONSTRAINT pk_frais_transport PRIMARY KEY (livraison, num),
    CONSTRAINT fk_frais_livraison FOREIGN KEY (livraison) REFERENCES Mode_Livraison(num)
);

CREATE TABLE Commande(
    num          INT NOT NULL AUTO_INCREMENT,
    destinataire VARCHAR(100) NOT NULL,
    prixTotal    DECIMAL(10,2) NOT NULL DEFAULT 0,
    livraison    INT NULL,           -- NULL : pas de frais de transport
    transport    INT NULL,
    CONSTRAINT pk_commande PRIMARY KEY (num),
    CONSTRAINT fk_commande_frais FOREIGN KEY (livraison, transport)
        REFERENCES Frais_Transport(livraison, num)
);

CREATE TABLE Ligne_Commande(
    commande INT NOT NULL,
    produit  INT NOT NULL,
    prixu    DECIMAL(8,2) NULL,      -- repris du produit s'il n'est pas fourni (Q5)
    quantite INT NOT NULL DEFAULT 1, -- nombre de participants pour une formation
    prixt    DECIMAL(10,2) NULL,     -- calculé par le trigger (Q5)
    CONSTRAINT pk_ligne_commande PRIMARY KEY (commande, produit),
    CONSTRAINT fk_ligne_commande FOREIGN KEY (commande) REFERENCES Commande(num),
    CONSTRAINT fk_ligne_produit  FOREIGN KEY (produit)  REFERENCES Produit(code),
    CONSTRAINT ck_ligne_quantite CHECK (quantite > 0)
);
```

Script de création des tables de l'exercice 2

Le jeu de données comprend deux formations, trois livres, deux modes de livraison avec leurs tranches de poids et quatre commandes : la 1 porte sur une formation et n'a pas de frais, la 2 contient des livres, la 3 est vide avec des frais et la 4 est vide sans frais. En le préparant, j'avais d'abord donné à la commande 2 la tranche 500-999 g alors que ses livres pesaient plus de 2 kg. Rien ne me l'a signalé, ce qui illustre un des cas non traités de la question 6. J'ai corrigé les quantités et les tranches pour que les données restent cohérentes.

ex2/03_jeu_de_donnees.sql — MySQL 9.7.4 – base tp5_commandes

Q1 – Jeu de données

```sql
-- Produits : formations (formation = 1, poids NULL) et livres (poids en g)
INSERT INTO Produit (code, libelle, prix, formation, poids) VALUES
    (1, 'Formation SQL avancé (2 jours)',        890.00, 1, NULL),
    (2, 'Formation Symfony initiation (3 jours)', 1250.00, 1, NULL),
    (3, 'Livre : Apprendre SQL',                  29.90, 0, 450),
    (4, 'Livre : PHP 8 et MySQL',                 39.50, 0, 820),
    (5, 'Livre : Bases de données relationnelles', 24.00, 0, 380);

-- Modes de livraison et frais de transport par tranche de poids (g)
INSERT INTO Mode_Livraison (num, libelle) VALUES
    (1, 'Colissimo'),
    (2, 'Chronopost');

INSERT INTO Frais_Transport (livraison, num, prix, poidsMin, poidsMax) VALUES
    (1, 1,  4.90,    0,  499),
    (1, 2,  6.90,  500,  999),
    (1, 3,  9.90, 1000, 4999),
    (2, 1, 12.50,    0,  999),
    (2, 2, 18.00, 1000, 4999);

-- Commandes : 1 = formation (sans frais), 2 = livres (Colissimo 1000–4999 g),
-- 3 = vide avec frais (Colissimo 500–999 g), 4 = vide sans frais
INSERT INTO Commande (num, destinataire, prixTotal, livraison, transport) VALUES
    (1, 'Entreprise Atlantis', 0, NULL, NULL),
    (2, 'Lucie Martin',        0, 1,    3),
    (3, 'Société Dupont',      0, 1,    2),
    (4, 'Karim Benali',        0, NULL, NULL);

-- Lignes : prixu omis -> repris du produit, prixt et prixTotal calculés
INSERT INTO Ligne_Commande (commande, produit, quantite) VALUES
    (1, 1, 3),   -- formation SQL pour 3 participants
    (2, 3, 1),   -- 1 livre SQL
    (2, 4, 1);   -- 1 livre PHP (450 + 820 = 1270 g)
```

Jeu de données de l'exercice 2

Client SQL – tp5_commandes — MySQL 9.7.4 – root@localhost

Q1 – Contenu des tables après le jeu de données

Produits : formations (formation = 1) et livres (formation = 0)

```
mysql> SELECT code, libelle, prix, formation, poids FROM Produit;
```

code	libelle	prix	formation	poids
1	Formation SQL avancé (2 jours)	890.00	1	*NULL*
2	Formation Symfony initiation (3 jours)	1250.00	1	*NULL*
3	Livre : Apprendre SQL	29.90	0	450
4	Livre : PHP 8 et MySQL	39.50	0	820
5	Livre : Bases de données relationnelles	24.00	0	380

5 lignes dans le jeu de résultats

Commandes : livraison / transport NULL = pas de frais

```
mysql> SELECT num, destinataire, prixTotal, livraison, transport FROM Commande;
```

num	destinataire	prixTotal	livraison	transport
1	Entreprise Atlantis	2670.00	*NULL*	*NULL*
2	Lucie Martin	69.40	1	3
3	Société Dupont	0.00	1	2
4	Karim Benali	0.00	*NULL*	*NULL*

4 lignes dans le jeu de résultats

Lignes de commande insérées par le jeu de données

```
mysql> SELECT commande, produit, prixu, quantite, prixt FROM Ligne_Commande;
```

commande	produit	prixu	quantite	prixt
1	1	890.00	3	2670.00
2	3	29.90	1	29.90
2	4	39.50	1	39.50

3 lignes dans le jeu de résultats

Contenu des tables après le jeu de données

Question 2 : pas de livre dans une commande de formation

Le trigger trg_lc_bi_ajout_livre se déclenche avant chaque insertion dans Ligne_Commande. Il lit d'abord, avec SELECT INTO, si le produit ajouté est une formation. Si c'est un livre (formation = 0), il

cherche avec EXISTS une ligne de la même commande dont le produit est une formation, et refuse l'ajout s'il en trouve une. Un trigger peut lire la table sur laquelle il porte ; c'est seulement la modifier qui est interdit.

ex2/02_triggers.sql — MySQL 9.7.4 – base tp5_commandes

Q2 – Pas de livre dans une commande qui contient une formation

```sql
-- ------------------------------------------------------------
-- Question 2 : si la commande porte déjà sur une formation, on
-- refuse l'ajout d'un livre. On lit d'abord le type du produit
-- ajouté, puis on cherche une formation parmi les lignes déjà
-- présentes dans la commande.
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_lc_bi_ajout_livre;

DELIMITER //
CREATE TRIGGER trg_lc_bi_ajout_livre
BEFORE INSERT ON Ligne_Commande
FOR EACH ROW
BEGIN
    DECLARE v_formation TINYINT;

    SELECT formation INTO v_formation
      FROM Produit
     WHERE code = NEW.produit;

    IF v_formation = 0 AND EXISTS (
        SELECT 1
          FROM Ligne_Commande lc
          JOIN Produit p ON p.code = lc.produit
         WHERE lc.commande = NEW.commande
           AND p.formation = 1
    ) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Ajout refusé : la commande porte sur une formation, elle ne peut pas contenir de livre.';
    END IF;
END //
DELIMITER ;
```

Trigger trg_lc_bi_ajout_livre (question 2)

L'ajout du livre 3 à la commande 1, qui contient la formation SQL, est refusé. L'ajout du livre 5 à la commande 2, qui ne contient que des livres, est accepté.

Client SQL – tp5_commandes — MySQL 9.7.4 – root@localhost

Q2 – Ajout d'un livre refusé dans une commande de formation

Avant : la commande 1 porte sur une formation

```
mysql> SELECT lc.commande, lc.produit, p.libelle, p.formation FROM Ligne_Commande lc JOIN Produit p ON p.code = lc.produit WHERE
       lc.commande = 1;
```

commande	produit	libelle	formation
1	1	Formation SQL avancé (2 jours)	1

1 ligne dans le jeu de résultats

Ajout du livre 3 à la commande 1

```
mysql> INSERT INTO Ligne_Commande (commande, produit, quantite) VALUES (1, 3, 1);
```

Erreur #1644 – SQLSTATE 45000
Ajout refusé : la commande porte sur une formation, elle ne peut pas contenir de livre.

Après : la commande 1 ne contient toujours que la formation

```
mysql> SELECT lc.commande, lc.produit, p.libelle, p.formation FROM Ligne_Commande lc JOIN Produit p ON p.code = lc.produit WHERE
       lc.commande = 1;
```

commande	produit	libelle	formation
1	1	Formation SQL avancé (2 jours)	1

1 ligne dans le jeu de résultats

Livre refusé dans une commande de formation

Client SQL – tp5_commandes — MySQL 9.7.4 – root@localhost

Q2 – Ajout d'un livre accepté dans une commande de livres

Avant : la commande 2 ne contient que des livres

```
mysql> SELECT lc.commande, lc.produit, p.libelle, p.formation FROM Ligne_Commande lc JOIN Produit p ON p.code = lc.produit WHERE
       lc.commande = 2;
```

commande	produit	libelle	formation
2	3	Livre : Apprendre SQL	0
2	4	Livre : PHP 8 et MySQL	0

2 lignes dans le jeu de résultats

Ajout du livre 5 à la commande 2

```
mysql> INSERT INTO Ligne_Commande (commande, produit, quantite) VALUES (2, 5, 1);
```

Requête OK – 1 ligne affectée

Après

```
mysql> SELECT lc.commande, lc.produit, p.libelle, p.formation FROM Ligne_Commande lc JOIN Produit p ON p.code = lc.produit WHERE
       lc.commande = 2;
```

commande	produit	libelle	formation
2	3	Livre : Apprendre SQL	0
2	4	Livre : PHP 8 et MySQL	0
2	5	Livre : Bases de données relationnelles	0

3 lignes dans le jeu de résultats

Livre accepté dans une commande de livres

Question 3 : contrainte réciproque

Le trigger trg_lc_bi_ajout_formation fait le contrôle inverse : si le produit ajouté est une formation et que la commande contient déjà un livre, l'ajout est refusé. Les deux triggers sont nécessaires, car chacun ne regarde que le type du produit qu'on ajoute : celui de la question 2 laisse toujours passer une formation.

ex2/02_triggers.sql — MySQL 9.7.4 – base tp5_commandes

Q3 – Pas de formation dans une commande qui contient un livre

```sql
-- ------------------------------------------------------------
-- Question 3 : contrainte réciproque. Si la commande contient déjà
-- un ou plusieurs livres, on refuse l'ajout d'une formation.
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_lc_bi_ajout_formation;

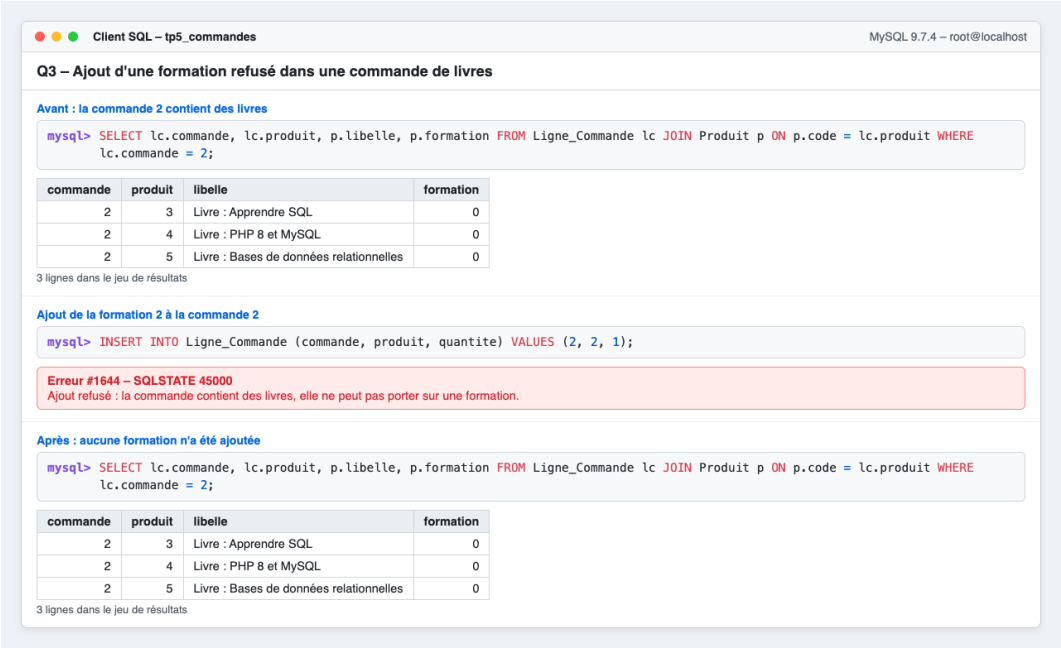
DELIMITER //
CREATE TRIGGER trg_lc_bi_ajout_formation
BEFORE INSERT ON Ligne_Commande
FOR EACH ROW
BEGIN
    DECLARE v_formation TINYINT;

    SELECT formation INTO v_formation
      FROM Produit
     WHERE code = NEW.produit;

    IF v_formation = 1 AND EXISTS (
        SELECT 1
          FROM Ligne_Commande lc
          JOIN Produit p ON p.code = lc.produit
         WHERE lc.commande = NEW.commande
           AND p.formation = 0
    ) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Ajout refusé : la commande contient des livres, elle ne peut pas porter sur une formation.';
    END IF;
END //
DELIMITER ;
```

Trigger trg_lc_bi_ajout_formation (question 3)

L'ajout de la formation 2 à la commande 2, qui contient trois livres, est refusé. L'ajout de cette formation pour deux participants à la commande 4, vide et sans frais, est accepté.



Client SQL – tp5_commandes — MySQL 9.7.4 – root@localhost

Q3 – Ajout d'une formation refusé dans une commande de livres

Avant : la commande 2 contient des livres

```
mysql> SELECT lc.commande, lc.produit, p.libelle, p.formation FROM Ligne_Commande lc JOIN Produit p ON p.code = lc.produit WHERE
       lc.commande = 2;
```

commande	produit	libelle	formation
2	3	Livre : Apprendre SQL	0
2	4	Livre : PHP 8 et MySQL	0
2	5	Livre : Bases de données relationnelles	0

3 lignes dans le jeu de résultats

Ajout de la formation 2 à la commande 2

```
mysql> INSERT INTO Ligne_Commande (commande, produit, quantite) VALUES (2, 2, 1);
```

Erreur #1644 – SQLSTATE 45000
Ajout refusé : la commande contient des livres, elle ne peut pas porter sur une formation.

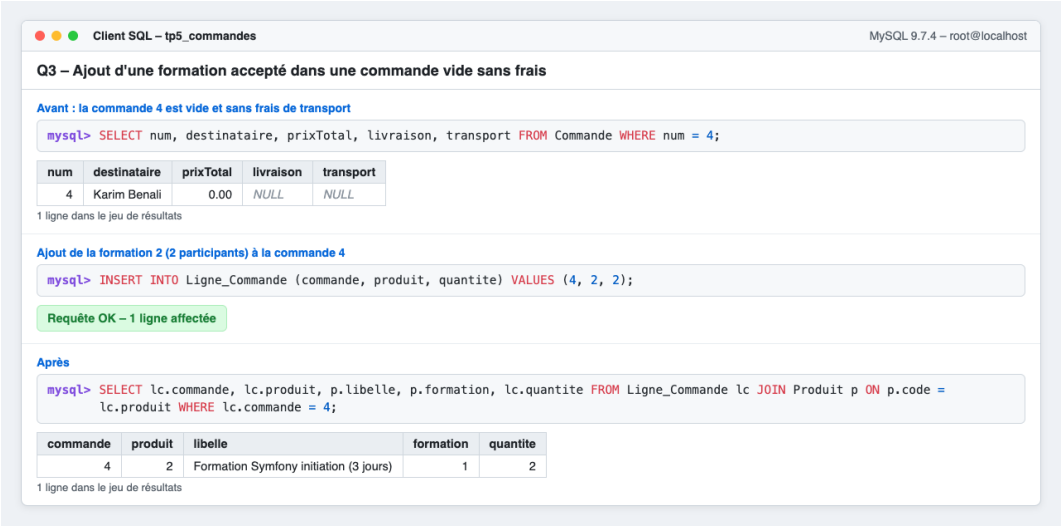
Après : aucune formation n'a été ajoutée

```
mysql> SELECT lc.commande, lc.produit, p.libelle, p.formation FROM Ligne_Commande lc JOIN Produit p ON p.code = lc.produit WHERE
       lc.commande = 2;
```

commande	produit	libelle	formation
2	3	Livre : Apprendre SQL	0
2	4	Livre : PHP 8 et MySQL	0
2	5	Livre : Bases de données relationnelles	0

3 lignes dans le jeu de résultats

Formation refusée dans une commande de livres



Client SQL – tp5_commandes — MySQL 9.7.4 – root@localhost

Q3 – Ajout d'une formation accepté dans une commande vide sans frais

Avant : la commande 4 est vide et sans frais de transport

```
mysql> SELECT num, destinataire, prixTotal, livraison, transport FROM Commande WHERE num = 4;
```

num	destinataire	prixTotal	livraison	transport
4	Karim Benali	0.00	*NULL*	*NULL*

1 ligne dans le jeu de résultats

Ajout de la formation 2 (2 participants) à la commande 4

```
mysql> INSERT INTO Ligne_Commande (commande, produit, quantite) VALUES (4, 2, 2);
```

Requête OK – 1 ligne affectée

Après

```
mysql> SELECT lc.commande, lc.produit, p.libelle, p.formation, lc.quantite FROM Ligne_Commande lc JOIN Produit p ON p.code =
       lc.produit WHERE lc.commande = 4;
```

commande	produit	libelle	formation	quantite
4	2	Formation Symfony initiation (3 jours)	1	2

1 ligne dans le jeu de résultats

Formation acceptée dans une commande vide sans frais

Question 4 : pas de frais de transport pour une formation

J'ai adapté le trigger de la question 3, puisque c'est lui qui traite déjà l'ajout d'une formation. Quand le produit est une formation, il vérifie maintenant aussi que la commande n'a pas de frais de transport, c'est-à-dire que livraison et transport sont NULL dans Commande :

```
SELECT 1 FROM Commande
 WHERE num = NEW.commande
   AND (livraison IS NOT NULL OR transport IS NOT NULL)
```

ex2/02_triggers.sql — MySQL 9.7.4 – base tp5_commandes

Q4 – Trigger de la Q3 adapté : pas de frais de transport pour une formation

```
-- ------------------------------------------------------------
-- Question 4 : on adapte le trigger de la Q3, qui traite déjà
-- l'ajout d'une formation. Une commande de formation ne doit pas
-- avoir de frais de transport : livraison et transport doivent
-- être NULL dans Commande, sinon l'ajout est refusé.
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_lc_bi_ajout_formation;

DELIMITER //
CREATE TRIGGER trg_lc_bi_ajout_formation
BEFORE INSERT ON Ligne_Commande
FOR EACH ROW
BEGIN
    DECLARE v_formation TINYINT;

    SELECT formation INTO v_formation
      FROM Produit
     WHERE code = NEW.produit;

    IF v_formation = 1 THEN
        -- (Q3) la commande ne doit pas contenir de livre
        IF EXISTS (
            SELECT 1
              FROM Ligne_Commande lc
              JOIN Produit p ON p.code = lc.produit
             WHERE lc.commande = NEW.commande
               AND p.formation = 0
        ) THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT =
                    'Ajout refusé : la commande contient des livres, elle ne peut pas porter sur une formation.';
        END IF;

        -- (Q4) la commande ne doit pas avoir de frais de transport
        IF EXISTS (
            SELECT 1
              FROM Commande
             WHERE num = NEW.commande
               AND (livraison IS NOT NULL OR transport IS NOT NULL)
        ) THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT =
                    'Ajout refusé : une commande de formation ne doit pas avoir de frais de transport.';
        END IF;
    END IF;
END //
DELIMITER ;
```

Trigger trg_lc_bi_ajout_formation adapté (question 4)

La commande 3 est vide mais a des frais Colissimo : l'ajout de la formation 1 y est refusé, avec le message sur les frais de transport. L'ajout d'une seconde formation à la commande 1, qui n'a pas de frais, est accepté.

Client SQL – tp5_commandes — MySQL 9.7.4 – root@localhost

Q4 – Formation refusée dans une commande avec frais de transport

Avant : la commande 3 est vide mais a des frais de transport (Colissimo, tranche 2)

```
mysql> SELECT num, destinataire, prixTotal, livraison, transport FROM Commande WHERE num = 3;
```

num	destinataire	prixTotal	livraison	transport
3	Société Dupont	0.00	1	2

1 ligne dans le jeu de résultats

Ajout de la formation 1 à la commande 3

```
mysql> INSERT INTO Ligne_Commande (commande, produit, quantite) VALUES (3, 1, 1);
```

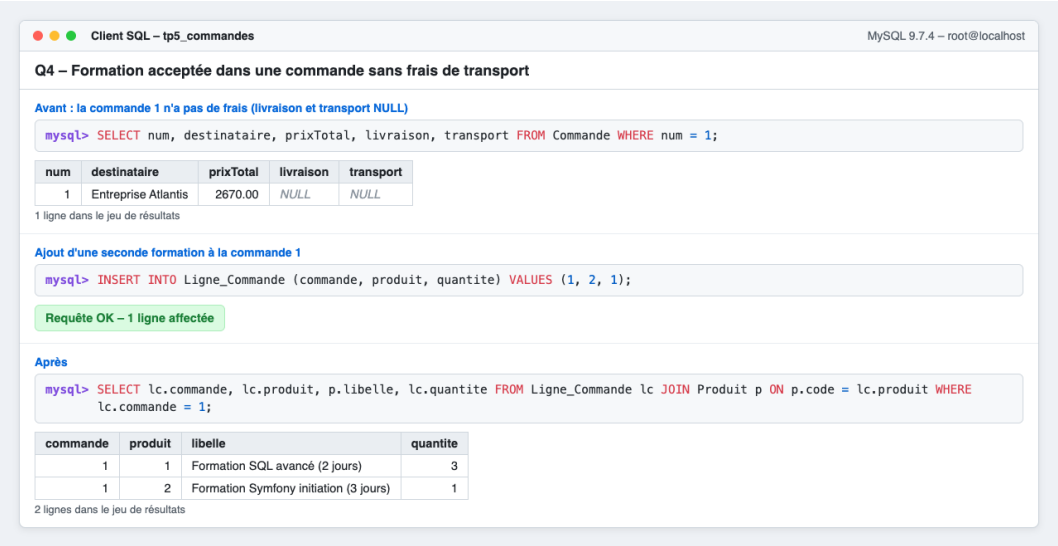
Erreur #1644 – SQLSTATE 45000
Ajout refusé : une commande de formation ne doit pas avoir de frais de transport.

Après : la commande 3 est toujours vide

```
mysql> SELECT commande, produit, prixu, quantite, prixt FROM Ligne_Commande WHERE commande = 3;
```

Jeu de résultats vide (0 ligne)

Formation refusée dans une commande avec frais de transport



Formation acceptée dans une commande sans frais

Question 5 : calcul du prix total

J'ai adapté le trigger de la question 2 pour qu'il calcule les prix. Si prixu n'est pas fourni, il reprend le prix catalogue du produit. Un prixu fourni, par exemple une remise, est conservé. Comme le prix d'un produit varie au fil du temps, la ligne garde ainsi le prix du jour de la commande. Le trigger calcule ensuite prixt = prixu * quantite, en modifiant NEW avant l'écriture, puis ajoute prixt au prixTotal de la commande. J'ai choisi de ne pas inclure les frais de transport dans prixTotal : ils dépendent du poids total, qui change à chaque ligne ajoutée, et le prix livraison comprise s'obtient en ajoutant Frais_Transport.prix.

J'ai recréé ce trigger avec FOLLOWS trg_lc_bi_ajout_formation pour qu'il s'exécute après l'autre trigger BEFORE INSERT : les calculs ne se font qu'une fois tous les contrôles passés. L'UPDATE de Commande est fait en BEFORE, mais si l'insertion échoue ensuite, InnoDB annule toute l'instruction, UPDATE compris.

ex2/02_triggers.sql — MySQL 9.7.4 – base tp5_commandes

Q5 – Trigger de la Q2 adapté : calcul du prix de la ligne et du prix total

```sql
-- ------------------------------------------------------------
-- Question 5 : on adapte le trigger de la Q2 pour calculer :
--  - prixu : repris du prix catalogue s'il n'est pas fourni ;
--  - prixt = prixu * quantite ;
--  - Commande.prixTotal = prixTotal + prixt.
-- Le prix total ne comprend que les produits : les frais de
-- transport (Frais_Transport.prix) ne sont pas inclus.
-- FOLLOWS : ce trigger s'exécute après trg_lc_bi_ajout_formation,
-- donc les calculs ne sont faits qu'une fois tous les contrôles
-- passés. Si l'INSERT échoue ensuite (doublon...), InnoDB annule
-- aussi l'UPDATE de Commande (l'instruction est atomique).
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_lc_bi_ajout_livre;

DELIMITER //
CREATE TRIGGER trg_lc_bi_ajout_livre
BEFORE INSERT ON Ligne_Commande
FOR EACH ROW
FOLLOWS trg_lc_bi_ajout_formation
BEGIN
    DECLARE v_formation TINYINT;
    DECLARE v_prix DECIMAL(8,2);

    SELECT formation, prix INTO v_formation, v_prix
      FROM Produit
     WHERE code = NEW.produit;

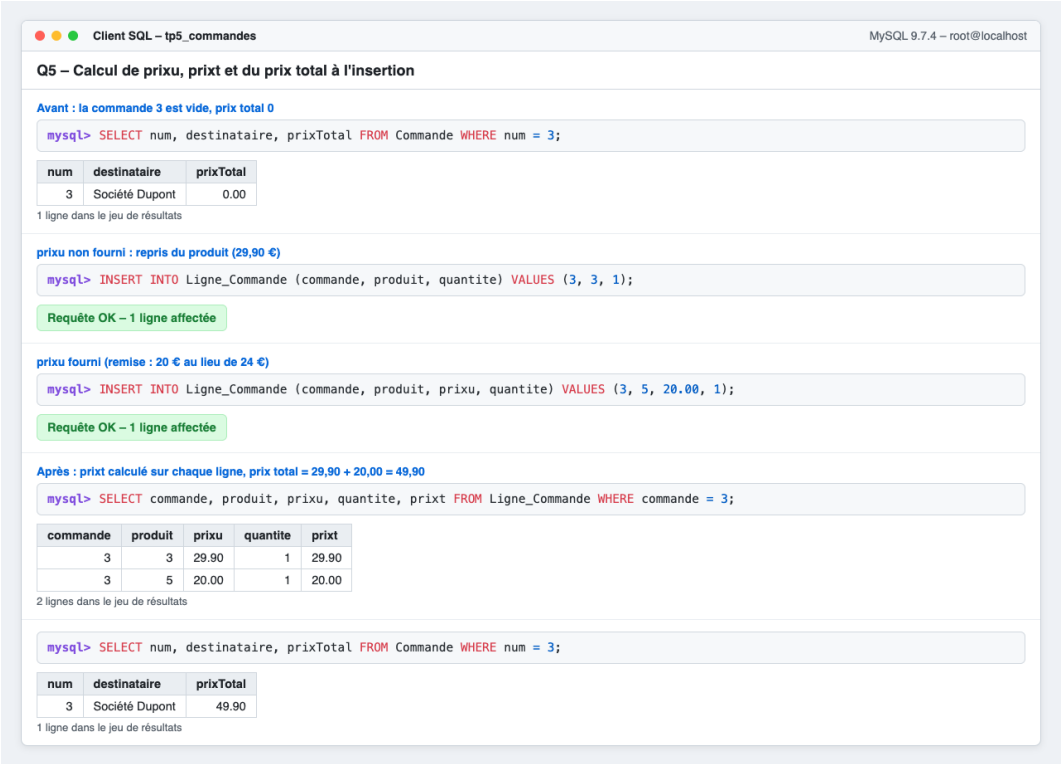
    -- (Q2) pas de livre dans une commande de formation
    IF v_formation = 0 AND EXISTS (
        SELECT 1
          FROM Ligne_Commande lc
          JOIN Produit p ON p.code = lc.produit
         WHERE lc.commande = NEW.commande
           AND p.formation = 1
    ) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Ajout refusé : la commande porte sur une formation, elle ne peut pas contenir de livre.';
    END IF;

    -- (Q5) prix de la ligne
    IF NEW.prixu IS NULL THEN
        SET NEW.prixu = v_prix;
    END IF;
    SET NEW.prixt = NEW.prixu * NEW.quantite;

    -- (Q5) prix total de la commande (hors frais de transport)
    UPDATE Commande
       SET prixTotal = prixTotal + NEW.prixt
     WHERE num = NEW.commande;
END //
DELIMITER ;
```

Trigger trg_lc_bi_ajout_livre adapté (question 5)

Sur la commande 3, vide, l'ajout du livre 3 sans prixu donne une ligne à 29,90 € repris du catalogue, et l'ajout du livre 5 avec un prixu de 20 € au lieu de 24 € donne une ligne à 20 €. Le prix total passe de 0 à 49,90 €. L'insertion d'une ligne en doublon est refusée par la clé primaire et le total reste à 49,90 €, ce qui confirme que l'UPDATE fait par le trigger a été annulé avec l'insertion. La requête de contrôle montre que, pour les quatre commandes, prixTotal est égal à la somme des prixt.



Client SQL – tp5_commandes — MySQL 9.7.4 – root@localhost

Q5 – Calcul de prixu, prixt et du prix total à l'insertion

Avant : la commande 3 est vide, prix total 0

```
mysql> SELECT num, destinataire, prixTotal FROM Commande WHERE num = 3;
```

num	destinataire	prixTotal
3	Société Dupont	0.00

1 ligne dans le jeu de résultats

prixu non fourni : repris du produit (29,90 €)

```
mysql> INSERT INTO Ligne_Commande (commande, produit, quantite) VALUES (3, 3, 1);
```

Requête OK – 1 ligne affectée

prixu fourni (remise : 20 € au lieu de 24 €)

```
mysql> INSERT INTO Ligne_Commande (commande, produit, prixu, quantite) VALUES (3, 5, 20.00, 1);
```

Requête OK – 1 ligne affectée

Après : prixt calculé sur chaque ligne, prix total = 29,90 + 20,00 = 49,90

```
mysql> SELECT commande, produit, prixu, quantite, prixt FROM Ligne_Commande WHERE commande = 3;
```

commande	produit	prixu	quantite	prixt
3	3	29.90	1	29.90
3	5	20.00	1	20.00

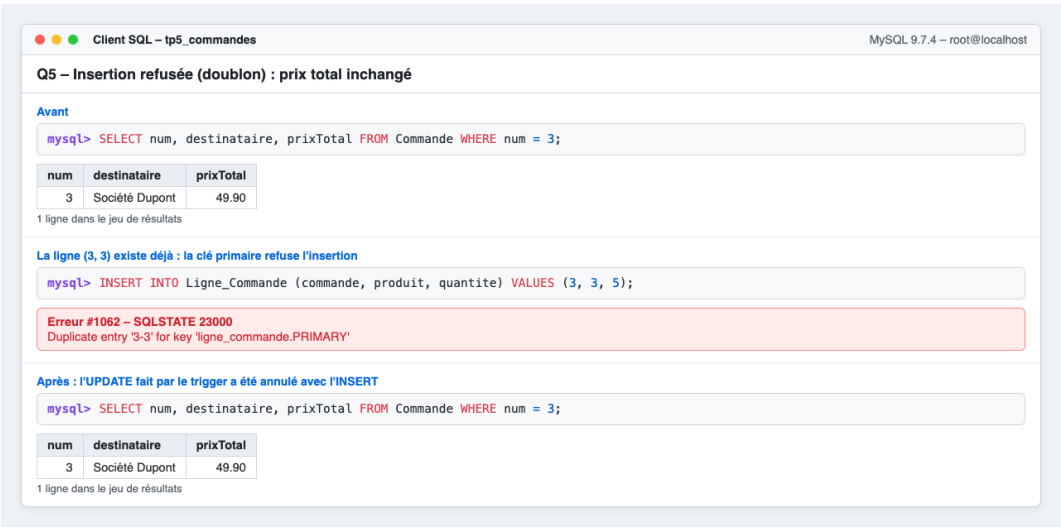
2 lignes dans le jeu de résultats

```
mysql> SELECT num, destinataire, prixTotal FROM Commande WHERE num = 3;
```

num	destinataire	prixTotal
3	Société Dupont	49.90

1 ligne dans le jeu de résultats

Calcul de prixu, prixt et du prix total



Client SQL – tp5_commandes — MySQL 9.7.4 – root@localhost

Q5 – Insertion refusée (doublon) : prix total inchangé

Avant

```
mysql> SELECT num, destinataire, prixTotal FROM Commande WHERE num = 3;
```

num	destinataire	prixTotal
3	Société Dupont	49.90

1 ligne dans le jeu de résultats

La ligne (3, 3) existe déjà : la clé primaire refuse l'insertion

```
mysql> INSERT INTO Ligne_Commande (commande, produit, quantite) VALUES (3, 3, 5);
```

Erreur #1062 – SQLSTATE 23000
Duplicate entry '3-3' for key 'ligne_commande.PRIMARY'

Après : l'UPDATE fait par le trigger a été annulé avec l'INSERT

```
mysql> SELECT num, destinataire, prixTotal FROM Commande WHERE num = 3;
```

num	destinataire	prixTotal
3	Société Dupont	49.90

1 ligne dans le jeu de résultats

Insertion refusée : prix total inchangé



Client SQL – tp5_commandes — MySQL 9.7.4 – root@localhost

Q5 – Vérification : prix total = somme des prixt de chaque commande

```
mysql> SELECT c.num, c.destinataire, c.prixTotal, COALESCE(SUM(lc.prixt), 0) AS somme_lignes,
              IF(c.prixTotal = COALESCE(SUM(lc.prixt), 0), 'OK', 'ÉCART') AS controle
         FROM Commande c LEFT JOIN Ligne_Commande lc ON lc.commande = c.num
        GROUP BY c.num, c.destinataire, c.prixTotal
        ORDER BY c.num;
```

num	destinataire	prixTotal	somme_lignes	controle
1	Entreprise Atlantis	3920.00	3920.00	OK
2	Lucie Martin	93.40	93.40	OK
3	Société Dupont	49.90	49.90	OK
4	Karim Benali	2500.00	2500.00	OK

4 lignes dans le jeu de résultats

Vérification : prix total égal à la somme des lignes

Question 6 : cas non traités

Les triggers des questions 2 à 5 ne portent que sur l'insertion d'une ligne de commande, donc toutes les autres opérations passent sans contrôle. Rien n'est prévu pour la mise à jour d'une ligne : un UPDATE Ligne_Commande SET produit = … permet de remplacer un livre par une formation et de contourner les questions 2, 3 et 4, et un changement de quantité ou de prixu ne recalcule ni prixt ni le prix total. Il faudrait un trigger BEFORE UPDATE qui refait les contrôles et met à jour le total avec prixTotal + NEW.prixt - OLD.prixt, en traitant le cas où la ligne change de commande. De même, la suppression d'une ligne laisse son montant dans prixTotal, alors qu'un trigger AFTER DELETE devrait retirer OLD.prixt.

Côté produits, le schéma ne garde aucun historique des prix. On ne peut donc pas savoir quel était le prix catalogue à une date donnée, ni vérifier qu'un prixu saisi à la main correspond à une remise autorisée. Si un produit change de nature (UPDATE Produit SET formation = 0 sur une formation déjà commandée), des commandes déjà acceptées deviennent mixtes ; il faudrait refuser ce changement dès que le produit figure dans une ligne. Rien ne vérifie non plus que le poids est NULL pour une formation et renseigné pour un livre.

Côté commande, la question 4 ne contrôle les frais qu'au moment où l'on ajoute une formation : un UPDATE Commande SET livraison = 1, transport = 1 sur une commande de formation passe sans erreur, ce qui demanderait un trigger BEFORE UPDATE sur Commande. La cohérence entre le poids total de la commande, c'est-à-dire la somme des poids multipliés par les quantités, et la tranche de Frais_Transport choisie n'est jamais vérifiée : mon erreur dans le jeu de données de la question 1 l'a montré. Il faudrait recalculer la tranche à chaque modification des lignes, ou refuser une ligne qui fait sortir la commande de sa tranche. Rien n'oblige non plus une commande de livres à avoir des frais de transport.

Enfin, MySQL ne vérifie pas une clé étrangère composée dès qu'une de ses colonnes est NULL : une commande avec livraison = 1 et transport = NULL est acceptée, alors qu'une contrainte CHECK (livraison IS NULL) = (transport IS NULL) l'empêcherait. Le champ prixTotal peut aussi être modifié directement par un UPDATE, alors qu'il devrait n'être calculé que par les triggers, comme la superficie de l'exercice 1. Les tranches de poids d'un même mode de livraison peuvent se chevaucher, poidsMin peut être supérieur à poidsMax, et aucun contrôle n'empêche un prixu négatif.

Exercice 3 : Location de véhicules

Question 1 : schéma relationnel et création des tables

Avant d'écrire le script, j'ai déduit le schéma relationnel du MCD :

Permis(num, type)
Vehicule(num, immatriculation, modele, #permis)
Conducteur(num, civilite, prenom, nom)
Titulaire(#conducteur, #permis)
Periode(debut, fin)
Location(#vehicule, #(debut, fin), #conducteur)

L'association « nécessiter » a une cardinalité 1,1 côté Véhicule : un véhicule exige un seul permis. Elle disparaît donc, et la clé de Permis devient la clé étrangère permis dans Vehicule, déclarée NOT NULL à cause du 1 minimum. « titulaire » est une association plusieurs à plusieurs, qui devient la table Titulaire

dont la clé primaire est formée des deux clés étrangères. Location, dessinée dans le cadre bleu, est une association entre Véhicule et Période regroupée en entité (agrégation) pour pouvoir participer elle-même à « louer ». Sa clé primaire est donc (vehicule, debut, fin), et un véhicule ne peut être loué qu'une fois sur une période donnée. « louer » a une cardinalité 1,1 côté Location : le conducteur devient une clé étrangère NOT NULL dans Location, mais il ne fait pas partie de la clé primaire, puisque la location est identifiée par le véhicule et la période. Période devient une table (debut, fin), à laquelle j'ai ajouté une contrainte CHECK debut < fin. J'ai aussi ajouté une colonne modele à Vehicule, à la place des « … » du MCD, pour rendre les données plus lisibles.

Le I relié à titulaire, nécessiter et louer est une contrainte d'inclusion : le permis exigé par le véhicule loué doit faire partie des permis dont le conducteur est titulaire. Des clés ne permettent pas d'exprimer cette règle, qui fait intervenir trois tables : c'est le rôle du trigger de la question 2.

ex3/01_creation_tables.sql — MySQL 9.7.4 – base tp5_location

Q1 – Script de création des tables (1/2)

```sql
DROP DATABASE IF EXISTS tp5_location;
CREATE DATABASE tp5_location CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE tp5_location;

CREATE TABLE Permis(
    num  INT NOT NULL AUTO_INCREMENT,
    type VARCHAR(5) NOT NULL,
    CONSTRAINT pk_permis PRIMARY KEY (num),
    CONSTRAINT uq_permis_type UNIQUE (type)
);

-- nécessiter (1,1) : la clé du permis exigé migre dans Vehicule
CREATE TABLE Vehicule(
    num             INT NOT NULL AUTO_INCREMENT,
    immatriculation VARCHAR(10) NOT NULL,
    modele          VARCHAR(50),
    permis          INT NOT NULL,
    CONSTRAINT pk_vehicule PRIMARY KEY (num),
    CONSTRAINT uq_vehicule_immat UNIQUE (immatriculation),
    CONSTRAINT fk_vehicule_permis FOREIGN KEY (permis) REFERENCES Permis(num)
);

CREATE TABLE Conducteur(
    num      INT NOT NULL AUTO_INCREMENT,
    civilite VARCHAR(4) NOT NULL,
    prenom   VARCHAR(50) NOT NULL,
    nom      VARCHAR(50) NOT NULL,
    CONSTRAINT pk_conducteur PRIMARY KEY (num)
);
```

Script de création des tables de l'exercice 3 (1/2)

ex3/01_creation_tables.sql — MySQL 9.7.4 – base tp5_location

Q1 – Script de création des tables (2/2)

```sql
-- titulaire (0,n – 0,n) : table d'association
CREATE TABLE Titulaire(
    conducteur INT NOT NULL,
    permis     INT NOT NULL,
    CONSTRAINT pk_titulaire PRIMARY KEY (conducteur, permis),
    CONSTRAINT fk_titulaire_conducteur FOREIGN KEY (conducteur) REFERENCES Conducteur(num),
    CONSTRAINT fk_titulaire_permis     FOREIGN KEY (permis)     REFERENCES Permis(num)
);

CREATE TABLE Periode(
    debut DATETIME NOT NULL,
    fin   DATETIME NOT NULL,
    CONSTRAINT pk_periode PRIMARY KEY (debut, fin),
    CONSTRAINT ck_periode CHECK (debut < fin)
);

-- Location : identifiée par le véhicule et la période,
-- louer (1,1) : le conducteur est une clé étrangère hors clé primaire
CREATE TABLE Location(
    vehicule   INT NOT NULL,
    debut      DATETIME NOT NULL,
    fin        DATETIME NOT NULL,
    conducteur INT NOT NULL,
    CONSTRAINT pk_location PRIMARY KEY (vehicule, debut, fin),
    CONSTRAINT fk_location_vehicule   FOREIGN KEY (vehicule)    REFERENCES Vehicule(num),
    CONSTRAINT fk_location_periode    FOREIGN KEY (debut, fin)  REFERENCES Periode(debut, fin),
    CONSTRAINT fk_location_conducteur FOREIGN KEY (conducteur)  REFERENCES Conducteur(num)
);
```

Script de création des tables de l'exercice 3 (2/2)

Le jeu de données comprend les permis A, B et C, quatre véhicules de catégories différentes (une moto, deux voitures et un poids lourd) et quatre conducteurs : Thomas n'a que le permis B, Sarah a les permis A et B, Julien les permis B et C, et Inès seulement le permis A.

ex3/03_jeu_de_donnees.sql — MySQL 9.7.4 – base tp5_location

Q1 – Jeu de données

```sql
INSERT INTO Permis (num, type) VALUES
    (1, 'A'),   -- moto
    (2, 'B'),   -- voiture
    (3, 'C');   -- poids lourd

INSERT INTO Vehicule (num, immatriculation, modele, permis) VALUES
    (1, 'AB-123-CD', 'Peugeot 208',       2),
    (2, 'EF-456-GH', 'Yamaha MT-07',      1),
    (3, 'IJ-789-KL', 'Renault Trucks D12', 3),
    (4, 'MN-012-OP', 'Renault Clio',      2);

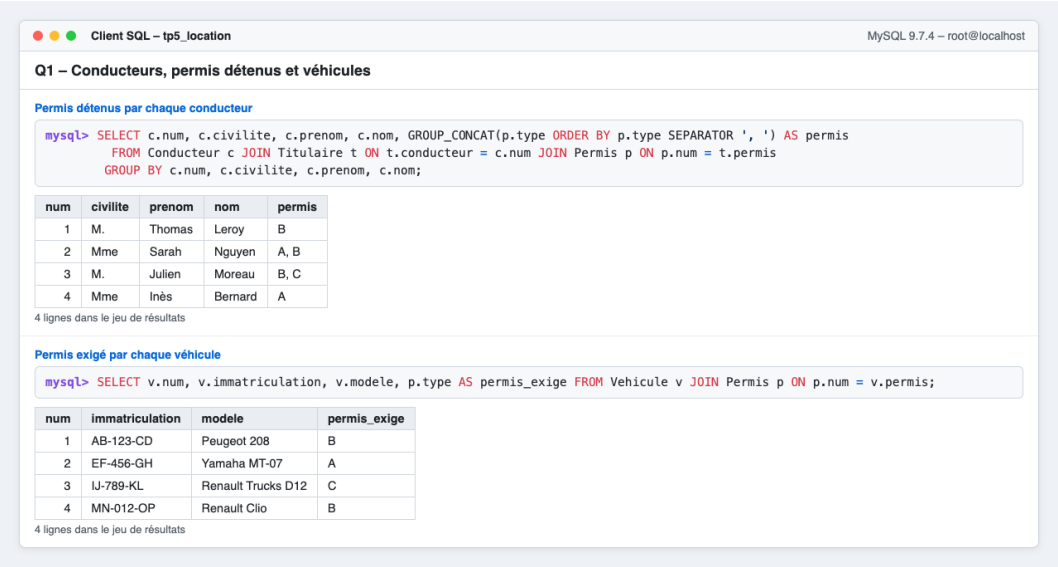
INSERT INTO Conducteur (num, civilite, prenom, nom) VALUES
    (1, 'M.',  'Thomas', 'Leroy'),
    (2, 'Mme', 'Sarah',  'Nguyen'),
    (3, 'M.',  'Julien', 'Moreau'),
    (4, 'Mme', 'Inès',   'Bernard');

-- Thomas : B ; Sarah : A et B ; Julien : B et C ; Inès : A seulement
INSERT INTO Titulaire (conducteur, permis) VALUES
    (1, 2),
    (2, 1), (2, 2),
    (3, 2), (3, 3),
    (4, 1);

INSERT INTO Periode (debut, fin) VALUES
    ('2026-11-02 08:00:00', '2026-11-04 18:00:00'),
    ('2026-11-09 08:00:00', '2026-11-10 18:00:00'),
    ('2026-11-16 08:00:00', '2026-11-20 18:00:00');

INSERT INTO Location (vehicule, debut, fin, conducteur) VALUES
    (1, '2026-11-02 08:00:00', '2026-11-04 18:00:00', 1),   -- Thomas, voiture (B)
    (2, '2026-11-02 08:00:00', '2026-11-04 18:00:00', 2),   -- Sarah, moto (A)
    (3, '2026-11-09 08:00:00', '2026-11-10 18:00:00', 3);   -- Julien, poids lourd (C)
```

Jeu de données de l'exercice 3



Permis détenus par les conducteurs et permis exigés par les véhicules

Question 2 : le conducteur doit avoir le permis exigé

J'ai écrit deux triggers BEFORE, l'un sur INSERT et l'autre sur UPDATE, avec le même contrôle. Le trigger récupère d'abord avec SELECT INTO le permis exigé par le véhicule de NEW, avec son type et l'immatriculation. Il vérifie ensuite avec NOT EXISTS que le couple (NEW.conducteur, permis) existe dans Titulaire. Pour que le message d'erreur soit utile, je le construis avec CONCAT dans une variable, puis je la passe à SIGNAL : il indique le conducteur, le permis manquant et le véhicule. Si le véhicule n'existe pas, v_permis reste NULL et je laisse la clé étrangère refuser la ligne ; sans ce test, CONCAT avec NULL donnerait un message vide.

ex3/02_triggers.sql — MySQL 9.7.4 – base tp5_location

Q2 – Le conducteur doit être titulaire du permis exigé (BEFORE INSERT)

```sql
-- ------------------------------------------------------------
-- Question 2 : à l'insertion et à la modification d'une location,
-- le conducteur doit être titulaire du permis exigé par le
-- véhicule (contrainte d'inclusion « I » du MCD). Un trigger ne
-- porte que sur un événement : même contrôle pour INSERT et UPDATE.
-- Le message d'erreur est construit avec CONCAT pour indiquer le
-- permis manquant. Si le véhicule n'existe pas, v_permis reste
-- NULL et c'est la clé étrangère qui refusera la ligne.
-- ------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_location_bi;
DROP TRIGGER IF EXISTS trg_location_bu;

DELIMITER //
CREATE TRIGGER trg_location_bi
BEFORE INSERT ON Location
FOR EACH ROW
BEGIN
    DECLARE v_permis INT;
    DECLARE v_type VARCHAR(5);
    DECLARE v_immat VARCHAR(10);
    DECLARE v_message VARCHAR(128);

    -- permis exigé par le véhicule loué
    SELECT v.permis, p.type, v.immatriculation
      INTO v_permis, v_type, v_immat
      FROM Vehicule v
      JOIN Permis p ON p.num = v.permis
     WHERE v.num = NEW.vehicule;

    -- le conducteur doit en être titulaire
    IF v_permis IS NOT NULL AND NOT EXISTS (
        SELECT 1 FROM Titulaire
         WHERE conducteur = NEW.conducteur
           AND permis = v_permis
    ) THEN
        SET v_message = CONCAT('Location refusée : le conducteur n°', NEW.conducteur,
                               ' n''a pas le permis ', v_type,
                               ' exigé par le véhicule ', v_immat, '.');
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_message;
    END IF;
END //
DELIMITER ;
```

Trigger trg_location_bi (question 2)

Pour le trigger de mise à jour, le contrôle porte sur NEW. Il couvre donc aussi bien un changement de conducteur qu'un changement de véhicule, puisque dans les deux cas c'est le nouveau couple conducteur et véhicule qui est vérifié.

ex3/02_triggers.sql — MySQL 9.7.4 – base tp5_location

Q2 – Même contrôle à la modification d'une location (BEFORE UPDATE)

```
-- ------------------------------------------------------------
-- Question 2 (modification) : même contrôle sur la ligne modifiée
-- (NEW), qu'on change le conducteur, le véhicule ou les deux.
-- ------------------------------------------------------------
DELIMITER //
CREATE TRIGGER trg_location_bu
BEFORE UPDATE ON Location
FOR EACH ROW
BEGIN
    DECLARE v_permis INT;
    DECLARE v_type VARCHAR(5);
    DECLARE v_immat VARCHAR(10);
    DECLARE v_message VARCHAR(128);

    SELECT v.permis, p.type, v.immatriculation
      INTO v_permis, v_type, v_immat
      FROM Vehicule v
      JOIN Permis p ON p.num = v.permis
     WHERE v.num = NEW.vehicule;

    IF v_permis IS NOT NULL AND NOT EXISTS (
        SELECT 1 FROM Titulaire
         WHERE conducteur = NEW.conducteur
           AND permis = v_permis
    ) THEN
        SET v_message = CONCAT('Modification refusée : le conducteur n°', NEW.conducteur,
                               ' n''a pas le permis ', v_type,
                               ' exigé par le véhicule ', v_immat, '.');
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_message;
    END IF;
END //
DELIMITER ;
```

Trigger trg_location_bu (question 2)

Les tests reprennent tous les cas. Sarah peut louer la moto (permis A) et Julien la Clio (permis B). Thomas ne peut pas louer la moto, et Inès ne peut pas louer le poids lourd : le message indique dans chaque cas le permis qui manque. En mise à jour, remplacer Thomas par Inès sur la Peugeot 208 est refusé, de même que donner le poids lourd à Sarah. Remplacer Thomas par Julien, qui a le permis B, est accepté.

Client SQL – tp5_location — MySQL 9.7.4 – root@localhost

Q2 – Location valide : le conducteur a le permis exigé

Avant : locations de la période du 16 au 20 novembre

```
mysql> SELECT vehicule, debut, fin, conducteur FROM Location WHERE debut = '2026-11-16 08:00:00';
```

Jeu de résultats vide (0 ligne)

Sarah (n°2, permis A et B) loue la moto EF-456-GH (permis A)

```
mysql> INSERT INTO Location (vehicule, debut, fin, conducteur) VALUES (2, '2026-11-16 08:00:00', '2026-11-20 18:00:00', 2);
```

Requête OK – 1 ligne affectée

Julien (n°3, permis B et C) loue la Clio MN-012-OP (permis B)

```
mysql> INSERT INTO Location (vehicule, debut, fin, conducteur) VALUES (4, '2026-11-16 08:00:00', '2026-11-20 18:00:00', 3);
```

Requête OK – 1 ligne affectée

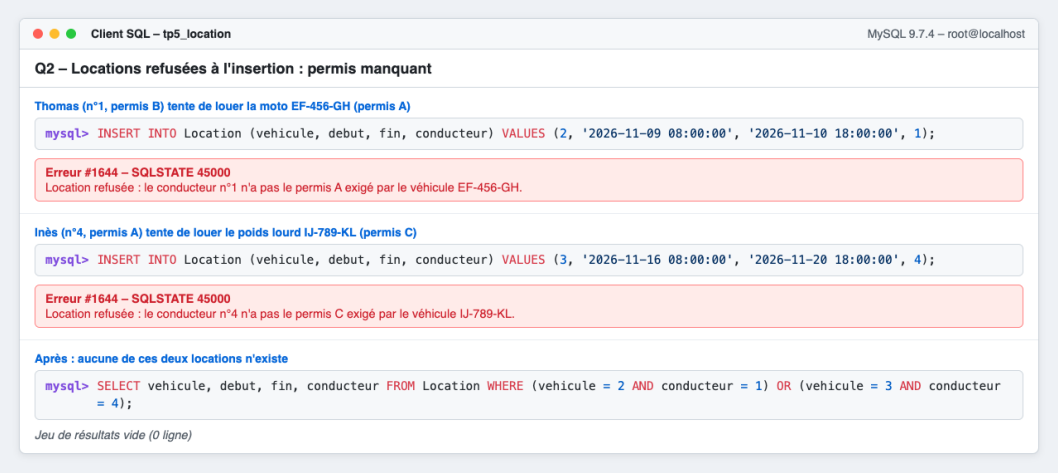
Après

```
mysql> SELECT vehicule, debut, fin, conducteur FROM Location WHERE debut = '2026-11-16 08:00:00';
```

vehicule	debut	fin	conducteur
2	2026-11-16 08:00:00	2026-11-20 18:00:00	2
4	2026-11-16 08:00:00	2026-11-20 18:00:00	3

2 lignes dans le jeu de résultats

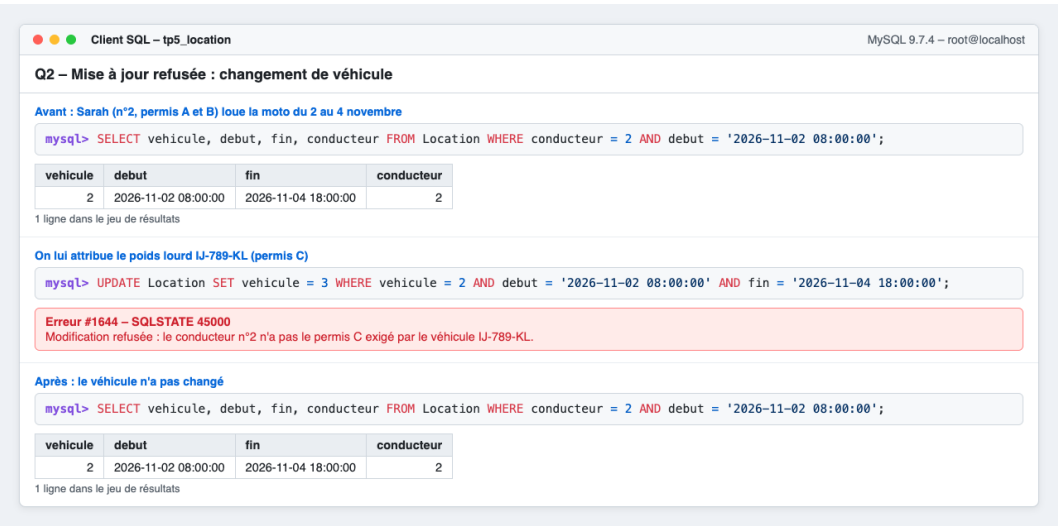
Locations valides acceptées



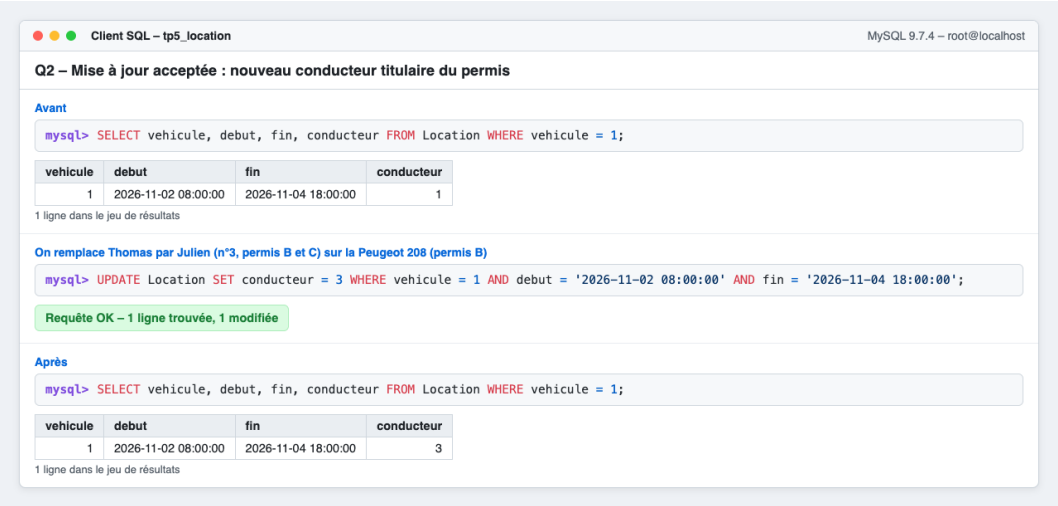
Locations refusées à l'insertion : permis manquant



Mise à jour refusée : changement de conducteur



Mise à jour refusée : changement de véhicule



Mise à jour acceptée : nouveau conducteur titulaire du permis B

Conclusion

Ce TP m'a permis de comprendre à quoi servent les triggers : faire respecter dans la base des règles que les clés ne peuvent pas exprimer. Dans l'exercice 1, ils vérifient la cohérence entre une place de parking et son prix, et ils maintiennent automatiquement une valeur calculée, la superficie d'un appartement. Dans l'exercice 2, ils interdisent de mélanger livres et formations dans une même commande et calculent le prix total. Dans l'exercice 3, ils font respecter une contrainte d'inclusion qui fait intervenir trois tables.

J'ai surtout retenu le choix entre BEFORE et AFTER : BEFORE pour refuser ou corriger la ligne, en modifiant NEW, et AFTER pour répercuter un changement validé sur une autre table. Je retiens aussi l'utilisation de NEW et OLD dans une mise à jour, en particulier quand une pièce change d'appartement. La principale difficulté a été le conflit entre les questions 3 et 4 de l'exercice 1 : le blocage de la superficie empêchait aussi les triggers de Piece de la mettre à jour. La variable de session que j'ai utilisée règle le problème, mais n'empêche pas un utilisateur de la positionner lui-même.

La question 6 de l'exercice 2 montre la limite de l'exercice : un trigger ne couvre que l'événement pour lequel il est écrit. Pour qu'une règle soit vraiment garantie, il faut la contrôler à l'insertion, à la mise à jour et à la suppression, sur toutes les tables concernées. Mon erreur de tranche de poids dans le jeu de données, que rien n'a détectée, en est un exemple concret.